Models Take Notes at Prefill: KV Cache Can Be Editable and Composable

Bojie Li
Pine AI

Code: https://github.com/19PINE-AI/programmable-kv
Website: https://01.me/research/programmable-kv/

Abstract

Prefix caching reuses prefill only across an exactly shared prefix, so one changed field invalidates the entire downstream cache. Yet overwriting the field's own key/value vectors and reusing the rest leaves the model acting on the *old* value. The reason, established causally across four model families: at prefill the model has already written the *field-conditioned conclusion* onto downstream *notes*; the field's own key/value drives under 1% of the decision. Attention does most of the writing, spread over many heads and tens of thousands of MLP neurons rather than a nameable few; a dozen late heads read the notes back additively. Read as a notebook of memoized conclusions, two capabilities follow. **(1) It is editable.** A salient *erratum* amends the notes; and with *chain-of-thought*, editing the field alone recovers the decision (1.00 at 8B, $\sim$1% compute), while without CoT it is ignored. **(2) It is composable.** The notes are position-portable, so a precompiled skill can be RoPE-repositioned and spliced into any context, indistinguishable from full recompute (logit cosine 0.90–0.999, twelve models) at $O(L)$ rather than $O(L^2)$ time-to-first-token. A unified edit+compose agent stays decision-identical to recompute at up to 14.9$\times$ lower latency. The approach applies to any per-token attention KV cache, validated across scale, quantization, Mixture-of-Experts, and multimodal caches, and extends to several attention variants through small adapters. Because the erratum is append-only, it composes with production prefix caching: in an online vLLM benchmark it keeps the prefix cache-aligned (98.5% hit-rate), cutting p90 time-to-first-token by 53–398$\times$.

1 Introduction

Modern LLM agents re-read long, mostly-static instructions on every turn—a system policy, a tool specification, retrieved documents. Key/value (KV) caching makes this affordable by reusing the prefill across turns, but only across an *exact* shared prefix. The moment one token changes inside the reused region—a timestamp, a user id, an order's status—the keys and values of *every* later token are invalidated, because each attended to the token that changed. The de-facto workaround—*hoisting* all mutable content to the end so the static prefix stays cache-aligned—pushes an inference-layer constraint into the application layer: fields referenced in several places, nested sub-agent prompts, and dynamically assembled contexts cannot all be cleanly hoisted, and the application must enumerate every mutable field in advance.

This paper starts from a concrete puzzle. The region of the cache *before* a field is, by construction, independent of the field's value—we measure a key/value deviation of exactly 0.0 when the field changes. One might therefore hope to *surgically* refresh only the field's own keys and values, leave the rest of the cache stale, and pay almost nothing. We find this fails completely: the model's decision reverts to the *old* field value, as if the edit never happened (Figure 1b).

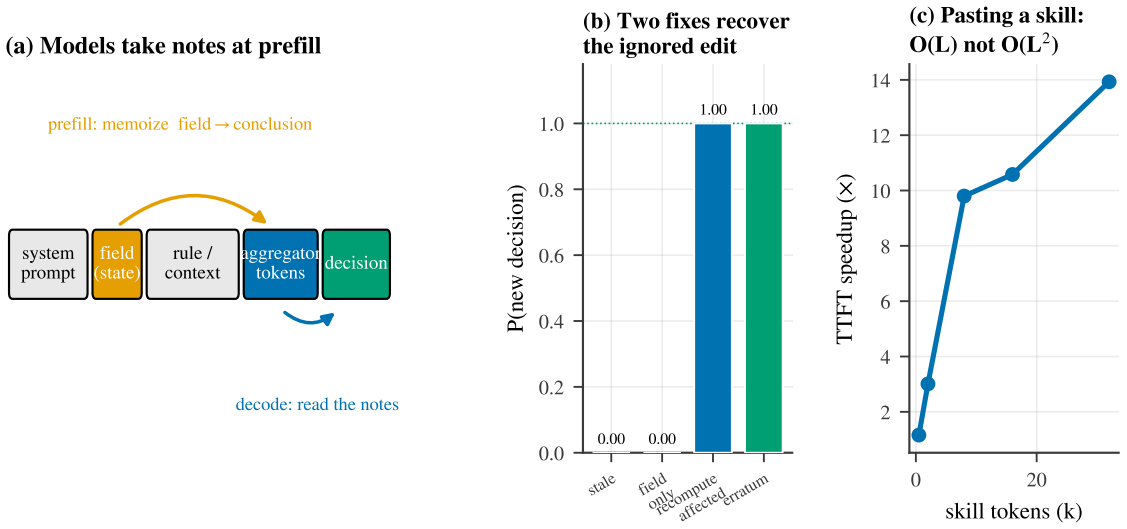


Figure 1: **Models take notes at prefill.** (a) At prefill the model memoizes the field-conditioned conclusion onto downstream aggregator tokens (orange); at decode the decision reads those notes (blue). (b) Consequently, surgically editing the field's own KV is ignored (without a reasoning chain), but the decision is recovered cheaply: recompute the *affected* downstream suffix, or—cheaper and robust— append a salient *erratum* (P(new decision), no-CoT model; with chain-of-thought the field-only refresh alone also works, Figure 2c). (c) Because the notes are position-portable, a precompiled skill can be pasted into a new context in $O(L)$ rather than $O(L^2)$ time.

The discovery. The reason, which we establish causally, is that transformers do not defer their reasoning to decode time. At *prefill* the model already computes the *field-conditioned conclusion* and writes it onto downstream tokens—disproportionately onto aggregator/delimiter tokens that later positions attend through. The decision then reads these *notes*, not the field itself: across models the field's own KV causally drives less than 1% of the decision, while the downstream notes drive essentially all of it. The KV cache is best understood not as a frozen byproduct of prefill but as a *notebook of memoized conclusions* (Figure 1a).

Two capabilities from one mechanism. Once the cache is read as a notebook of memoized conclusions, we can manipulate those conclusions directly—editing or reusing them in place rather than recomputing them—in two ways. (1) If the conclusion is already written downstream, then *editing* a field means amending the notes, not recomputing them. The cheap and robust fix is a one-line salient *erratum* that overrides the stale notes; one can instead recompute the affected downstream notes (reliably for the full affected suffix, cheaply-but-unreliably for a top-K, `field+selective@`K). An even cheaper in-place refresh of the field alone is gated by *chain-of-thought*: with a reasoning chain the model re-reads the field and the edit suffices, without it the edit is ignored. (2) If the notes are localized and position-portable, then a reusable skill can be *composed* into a new context by repositioning and splicing its cached notes—no recompute. A *unifying* experiment, editing a field *inside* a transplanted skill, shows the two operations act on the same notes. Figure 2 previews both capabilities. More broadly, we view editing and composition as first instances of a *programmable* KV cache—a structured memory that systems, and eventually models trained to expose it, can read, write, and rearrange rather than only extend linearly.

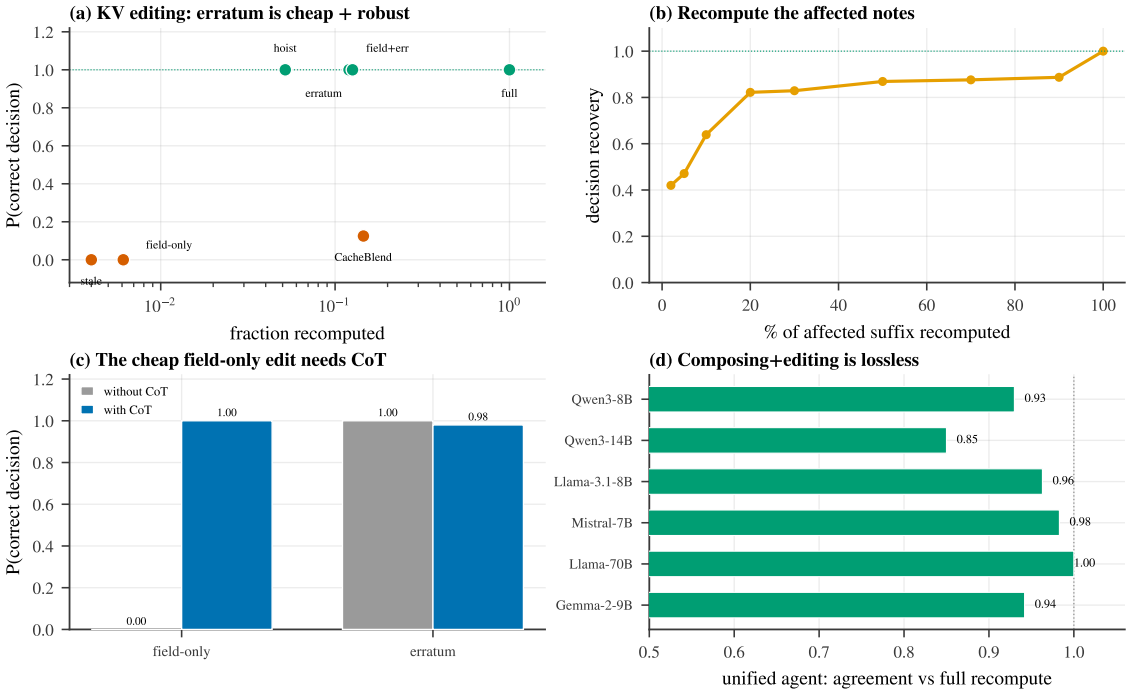


Figure 2: **Editing and composing, previewed** (Qwen3-8B unless noted; detail in Sections 3, 4 and 6). (a) *KV editing landscape*: naive edits (stale, field-only without CoT, CacheBlend) fail, while the append-only `erratum`/`field+erratum` reach full-reprefill correctness cheaply and robustly (hoisting also works but needs prompt surgery). (b) *Recompute the affected notes*: recovery climbs as more of the post-field *affected suffix* is recomputed—reliable in the limit, but the cheap top-K version (`field+selective@`K) is unreliable (Figure 5). (c) *Chain-of-thought, not model size, gates the cheapest edit*: with CoT the near-free field-only refresh alone recovers the decision (1.00); without CoT it is ignored (0.00); the erratum fixes both. (d) *Composing + editing is lossless*: the unified agent stays decision-identical to full recompute across models.

Structure of the paper.

- **A mechanism** (Section 3)—*attention-mediated memoized inference*—established by four causal probes (locality patching, suffix concentration, linear probing, circuit knockout) and four further controls (content/conclusion dissociation, layer-timing, specificity, note-injection; Appendix C), replicated across *four model families* (Qwen3, Llama-3.1, Gemma-2, Mistral), resolved to a *component-level circuit*—named read/write heads, a causal conclusion direction, an SAE feature, attention-vs-MLP, and causal scrubbing (Appendix D)—and down to individual MLP neurons, where the write is diffuse and neuron labels that read cleanly fail to predict what the neurons do (Appendix E); the account connects to the delimiter-token aggregation seen in interpretability.
- **An editing capability** (Section 4): naive KV editing fails; the erratum / `field+erratum` fix matches the hoist-to-end oracle without prompt surgery; an analysis of when the $\sim$1%-compute in-place edit suffices (it requires reasoning and is strongly model-dependent); and a head-to-head with *weight* editing (ROME, LoRA) showing it is the wrong tool for mutable per-request state (global contamination, collateral, 30–50$\times$ slower).
- **A composing capability** (Section 5): position-portable transplant of precompiled skills, $O(L)$

vs. $O(L^2)$ TTFT (13.9× at 32k), with a seam-repair knob—building on prior caching work (Section 2), our addition being the mechanism that explains it and a correctness lens.

- **The unification** (Section 6): this experiment and a unified edit+compose agent over thirteen models.
- **An application: user memory** (Section 7): the large, mutable user-memory document is both composed (precompiled, repositioned, spliced) and edited (in-place / erratum) as one set of notes—decision-faithful to full recompute at 2.3–4.3× lower time-to-first-token, validated to 70B and on real long-conversation memory (LoCoMo, transplant ≡ full recompute in QA accuracy)—with a pre-registered, statistically-controlled evaluation.
- **Applicability to multimodal and new attention mechanisms** (Section 8): the mechanism carries from small models through MoE and low-bit quantization to multimodal image caches; lightweight adapters bring it to MLA and interleaved M-RoPE; a mask-based remedy handles sliding-window attention; and we chart where it breaks down against the 2026 sparse/compressed-attention frontier.
- **Systems payoff** (Section 9): a real agentic environment and a comprehensive online vLLM serving benchmark (V1 engine, continuous batching, Poisson load)—98.5% vs. 1% prefix-cache hit-rate, 53–398× lower $p90$ TTFT, and throughput gains that grow with load to 14.5×.

2 Related work

Where computation is stored, and how to edit it. A line of interpretability work localizes stored *knowledge* in transformer weights and edits it there: ROME and MEMIT [25, 26] locate and rewrite factual associations in MLP weights, while circuit analyses such as the indirect-object-identification study [35] trace how specific computations are carried by attention heads. Weight editing targets *durable, global* facts; we compare against a faithful ROME and a LoRA fine-tune empirically (Table 1) and find them ill-suited to *mutable per-request* state—a global edit contaminates concurrent requests and damages unrelated decisions—which is exactly the niche the editable KV cache fills. Closest in spirit, Lindsey et al. [20] find models commit *plans* to specific tokens (e.g. a planned rhyme stored on a line-break token) during the forward pass. We study a complementary object: not weights and not decode-time computation, but the *KV cache*—the activations an inference system already stores and an editor can directly manipulate—and show it holds *memoized conclusions* concentrated on aggregator/delimiter tokens. To our knowledge this is the first causal account of why in-place KV field editing fails and what to do instead.

KV reuse and composable caching. Reusing precomputed KV beyond an exact prefix is an active systems topic, and our *composing* capability builds directly on it; we claim none of the caching machinery as novel. Prompt Cache [9] precomputes reusable prompt modules with position placeholders and splices them. CacheBlend [38] reuses non-prefix chunk KV and selectively recomputes ∼15% of tokens to restore cross-attention. EPIC [13] introduces position-independent caching with AttnLink, recomputing only a few chunk-boundary tokens (exploiting attention sinks) for near-linear recompute. CacheSlide [21] reuses KV in a position-*aware* way via relative-position-dependent caching, and MPIC [43] extends position-independent caching to the multimodal setting by recomputing image-boundary tokens; KVLink [37] is a further reuse system. Mapped onto our terms, our RoPE-repositioning is CacheSlide's relative-position reuse, our seam-repair is the boundary recompute of CacheBlend/EPIC/MPIC, and our image-KV transplant is MPIC's idea (we *re-rotate* M-RoPE rather than recompute). Our contributions over this line are orthogonal: (i) the *mechanism* that explains *why* boundary recompute is what is needed; (ii) a *decision-governance*

evaluation—does a transplanted skill still *govern the tool decision*, rather than only preserve perplexity or throughput; (iii) the *editing* axis and the edit+compose unification; and (iv) adapters that extend the operations to new attention representations (MLA, interleaved M-RoPE, sliding-window). We note honestly that this editable/composable-cache direction grew directly out of the position-independent and position-aware caching of EPIC and CacheSlide [13, 21], and out of discussions with Junhao Hu (first author of EPIC; see Acknowledgements).

Prefix caching, KV compression, and reuse systems. Production prefix caching (vLLM Automatic Prefix Caching, SGLang RadixAttention) reuses exact prefixes. A large literature instead *compresses* or *evicts* the cache: StreamingLLM keeps attention sinks [36]; H2O [42], Scissorhands [23], and SnapKV [19] evict low-importance tokens; Quest [32] keeps all tokens but attends sparsely. Serving systems stream or share cached KV across requests—CacheGen [22] for fast loading and RAGCache [16] for retrieval reuse. These change *which* tokens are present, and compose with our edit/transplant operations only over retained tokens; we treat them, the latent/decoupled-RoPE representation of Multi-head Latent Attention [3, 4], and the 2026 sparse/compressed-attention designs as scope boundaries in Section 8. Our repositioning relies on rotary position embeddings [31] and their extensions [28].

Activation-level interventions. Beyond weight editing, a body of work intervenes on *activations*: steering and inference-time interventions [18], task and function vectors [14, 33], and the causal-mediation/activation-patching methodology we adapt [34], with attribution patching as its first-order approximation [17]. At the level of single units, LLM-written neuron explanations are scored by how well activations simulated from the explanation match the real ones [2]; we use the stricter substitution form of that test to check whether legible labels explain the neurons that write our notes (Appendix E). Steering methods edit residual-stream directions to change behavior; we instead read and write the *KV cache* itself—the per-token activations a serving system already persists—which is what makes the intervention both interpretable and deployable.

Agents and tool use. Our evaluation targets tool-using agents in the style of ReAct [39] and Toolformer [30], and we measure end-to-end task success on the τ^2-bench tool-agent-user benchmark [1], the dual-control successor of τ-bench [40], where state changes mid-trajectory make cache staleness a first-class correctness problem.

3 The discovery: memoized inference in the KV cache

Where the conclusion is written and read. We first state the account that the probes in this section establish. A transformer prefills the prompt left-to-right, each position attending to all earlier ones. At a small number of positions *after* the mutable field—*aggregator* tokens such as punctuation, newlines, and section breaks, which later tokens route attention through—the model does more than cache the raw token: it computes the field-conditioned answer (e.g. "status is `shipped`, so the action is *deny*") and writes that conclusion into those positions' cached key/value vectors at mid-to-late layers. These cached vectors are the *notes*, and they lie *downstream* of the field. At decode time the decision token does not re-derive the answer from the field; it attends back to the note positions, and a small set of late-layer attention heads read the stored conclusion into the output logit. Refreshing the field's *own* KV therefore changes little, since the conclusion was written elsewhere and the field itself accounts for under 1% of the decision. The four probes below establish each link of this chain causally: *where* the notes lie, that they encode a *conclusion* rather than a copy of the field, and that the decision *reads* them.

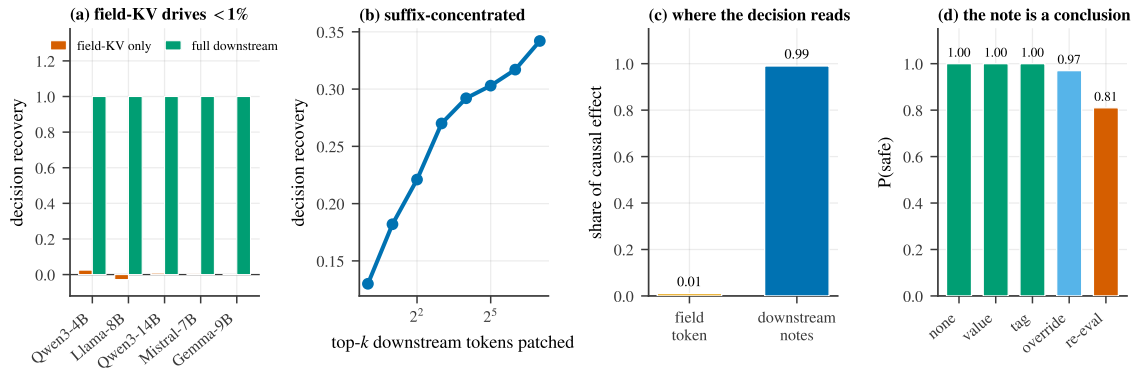


Figure 3: **Four causal probes for memoized inference.** (a) Refreshing the field's own KV recovers ≈ 0 of the decision; recomputing the downstream recovers it fully. (b) Recovery is suffix-concentrated, accruing only as many post-field tokens are patched. (c) The decision reads almost entirely from downstream notes, not the field token. (d) Once the value is present, override wording is redundant and "re-evaluate" phrasing hurts—the note is a committed conclusion.

Setup. We study a minimal but agent-realistic decision: a context contains a policy rule and a *mutable field* whose value determines the correct action (e.g. "cancel an order only if its status is `pending`"; with `status=shipped` the correct action flips from *cancel* to *deny*). After prefilling the full context we compare four cache states at the decision token: *stale* (old field, old downstream), *field-only* (refresh the field's KV, leave the downstream stale), *full-downstream* (recompute everything after the field), and *oracle* (a clean prefill of the new value). We report *decision recovery*: the fraction of the oracle's flip that a cache state reproduces (0 = behaves like stale, 1 = behaves like oracle). Four independent probes converge on one account (Figure 3); Appendix B gives a complete worked example—the verbatim prompt, the erratum, and recorded model responses.

(1) Locality: the field's own KV barely matters. Refreshing only the field's KV recovers essentially none of the decision—field-only recovery is -0.028 on Llama-3.1-8B and near zero across models—whereas recomputing the downstream recovers it fully (1.0) (Figure 3a). The field is read *indirectly*: its causal contribution to the decision is under 1%.

(2) Suffix concentration: the effect lives downstream and late. Sweeping how many downstream tokens we patch (ranked by causal effect) shows recovery accrues slowly and saturates only as we include many tokens after the field, with the causal mass concentrated in mid/late layers (Figure 3b). The information the decision needs is not in the field but distributed over the tokens that followed it at prefill (Figure 3c).

(3) Linear decodability. The field-conditioned conclusion is linearly decodable from those downstream tokens' residual stream at prefill time—i.e. the model has already *computed and written down* the answer, not merely copied the field.

(4) Knockout and dose-response. Ablating the high-effect downstream tokens flips the decision back to stale, and the effect grows with the number/position of memoizing tokens, ruling out a positionally diffuse explanation: specific aggregator/delimiter positions carry the conclusion. This mirrors the delimiter-token aggregation reported by Lindsey et al. [20] for forward planning; here the stored quantity is a backward-looking, field-conditioned *conclusion*.

What the note contains. If the note were a verbatim copy of the field, restating the value would suffice and emphatic wording would be inert. Instead, a wording ablation (Figure 3d) shows that once the corrected value is present, the override phrasing is *redundant* (bare value, tagged update, and explicit override all reach ≈ 1.0 P(safe)), while aggressive "disregard your earlier conclusion and re-evaluate" phrasing actively *hurts* (0.81). The note behaves like a committed conclusion that a late, salient correction can overwrite—but that confrontational instructions can destabilize. We name the phenomenon **attention-mediated memoized inference**. The remainder of this section stress-tests the account—further controls, more architecture families, and a component-level circuit—before the rest of the paper builds on it.

Separating the stored conclusion from the field content. A linear probe can *decode* the conclusion from the downstream tokens, but decodability alone does not show the decision *uses* it—the same tokens also encode the field's raw content, and a probe finds both. Four causal controls (three models each: Qwen3-8B/4B and Llama-3.1-8B; Appendix C) close that gap; the causal patch, not the probe, is the instrument that separates the two.

(i) It is the conclusion, not the field content. We hold the field value byte-identical and flip a single rule token (a polarity *trigger*) so the *conclusion* inverts while the field *content* stays constant. Transplanting the downstream notes then carries the entire flipped conclusion (recovery 0.998–1.009), whereas patching the changed rule token carries none (-0.007 to $+0.007$). Since the content never changed, the notes cannot merely be re-encoding it.

(ii) The note is written before it is read. Within the single prefill pass the conclusion becomes decodable on the downstream aggregator about twelve layers *earlier* (relative depth 0.31–0.39) than the point at which the decision token commits to its answer (depth 0.73–0.77): the note already exists at prefill, ahead of the read.

(iii) A few specific tokens carry it. Transplanting the eight *highest-effect* downstream positions recovers 0.74–0.79 of the decision, while eight *random* downstream positions recover ≤ 0.035: the conclusion lives in a few specific aggregator tokens, not spread over the whole downstream. (Within those tokens the code is diffuse across neurons; see the circuit paragraph below.)

(iv) The decision follows the note, even when the note is wrong. Injecting a *false* note (the opposite conclusion's downstream KV) into an otherwise-consistent cache makes the decision follow the written note *against its own live field* (recovery ≈ 1.0); a handful of note tokens suffice. The account also holds off the synthetic template: field-only recovery stays ≈ 0 for multi-hop reasoning and free-form conversational phrasing, and is bounded only for near-verbatim attribute lookup, where the field is itself partly a copy (Appendix C).

Not a model-family artifact: four families. To rule out the aggregator-token account being a Qwen3/Llama tokenizer artifact, we replicate five probes (the locality probe and the four deep controls) on two further architecture families, **Gemma-2-9B** and **Mistral-7B** (a tokenizer-robust readout; for Gemma-2 we keep its attention/logit soft-capping intact). Every result holds: field-only recovery 0.005/0.137 vs. full-downstream 1.0; the conclusion/content dissociation (trigger-only ≈ 0 with the field held identical vs. notes ≈ 1.0); top-8 vs. random-8 specificity (0.95 vs. 0.48); false-note injection (0.98–1.0, follow-rate 1.0); and write-before-read timing (write depth 0.19–0.26 vs. decision commit 0.47–0.48). The mechanism is consistent across *four* families (Qwen3, Llama-3.1, Gemma-2, Mistral; Appendix C).

A component-level circuit. The probes above show *where* the conclusion is stored; five further interventions (Appendix D; replicated across Llama-3.1, Qwen3, Gemma-2, and Mistral) show *which components* write and read it. The pattern is *distributed write, concentrated read*.

Write (many components, mid layers). Mid-layer *attention* does most of the writing onto the

aggregator tokens. On Llama it contributes ≈ 0.6 of the direct write, and patching every attention output at the top-8 note positions restores 0.89 of the decision, against 0.39 for every MLP output. The conclusion is stored *redundantly*. Several write heads each hold an overlapping copy: their single-head sufficiencies sum to twice their joint effect. The conclusion is also easy to *decode* (a trained sparse-autoencoder feature separates the two conclusions at AUC 1.0), yet its causal content is spread across $\sim$10–30 features along a shared low-rank direction (that direction carries $\approx 25\times$ the decision-effect of a random one).

Read (few components, late layers). A small, nameable set of late *read heads*—those attending from the decision token back to the aggregators—funnels the stored conclusion into the output logit (12 such heads recover 0.78 of the decision; the same number of random heads recover ≈ 0). The read heads act additively and independently. Ablating all twelve together removes 0.49 of the decision, close to the 0.52 sum of their single-head ablations, and none of them takes over when another is removed.

Neurons: legible, but neither few nor predictable. At the finest grain (Appendix E; Llama-3.1-8B, 458,752 MLP neurons), the MLP share of the write is diffuse. The ten neurons with the largest attribution carry under 1% of the total, and half of it needs $\sim$28k neurons ($\sim$27k on Qwen3-8B). Patching the top 30 neurons recovers 0.30 of the decision and the top 300 recover 0.72; random neurons recover ≈ 0. Negative activations matter in aggregate (zeroing every negative activation at the note positions removes 0.55 of the decision, every positive one 0.70), but the eight most important negative-side neurons remove only 0.011. An LLM labeller gives 96 of the top 100 neurons a coherent one- or two-part label. Yet these labels fail *Gao's simulation–substitution test* [7]: replacing a neuron's real activations with ones an LLM predicts from its label should preserve its effect if the label explains it. Simulated activations of the top 30 recover 0.03 of the decision, against 0.30 for the real ones. A neuron label can read cleanly without explaining what the neuron does.

Control. Causal scrubbing confirms the note alone governs the decision: resampling it to the opposite conclusion flips the decision, while resampling everything else leaves it unchanged. As at the token level, the single feature that best *decodes* the conclusion is not by itself *causally* sufficient—decodability and causation come apart, now among individual features and, as the neuron labels show, between how legible a unit is and what it does. With the mechanism established, the rest of the paper turns it into capabilities: the cache is editable (Section 4) and composable (Section 5).

4 Consequence I: the cache is editable

The mechanism tells us how to repair the cache after a field changes, not only why a naive edit fails. Because the decision reads a conclusion stored in the downstream notes (Section 3), *editing* means amending those notes rather than recomputing the prefix; this section establishes when a naive edit fails, the two ways to repair the notes, and which to prefer. Figure 4 contrasts the two cache operations this section and the next make precise: editing appends a salient correction; composing repositions and splices precompiled KV.

Naive editing fails—by design. The mechanism predicts the in-place edit will fail, and it does: refreshing the field's KV while reusing the stale downstream leaves the decision at the old value (Figure 1b). The prefix before the field is genuinely reusable (deviation 0.0); the problem is that the conclusion was already memoized after it.

Two ways to fix it, neither a full reprefill. If the stale notes carry an old conclusion, there are two interventions. *(i) Recompute the affected notes* after the field. The safe version recomputes the *whole* suffix after the field—exactly the *full-downstream* state from Section 3 (recov-

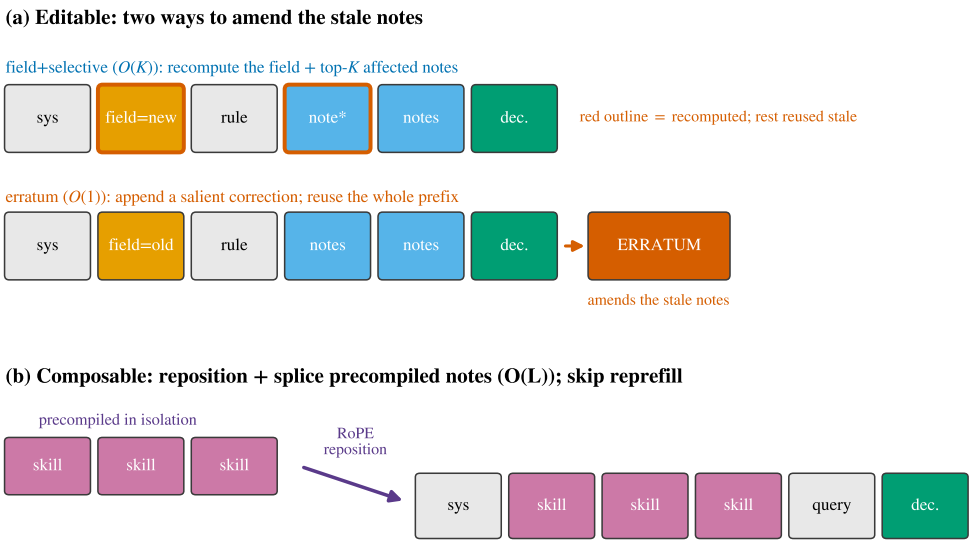


Figure 4: **Two cache operations.** (a) *Editable*, two ways to amend the stale notes: `field+selective` recomputes the field plus the top-K highest-effect downstream notes ($O(K)$; recomputed cells outlined in red, the rest reused stale), while the `erratum` appends one salient correction ($O(1)$, the whole prefix is reused). (b) *Composable*: precompute a skill's KV in isolation, RoPE-reposition it to the target positions, and splice it in ($O(L)$, no reprefill).

ery 1.0)—but pays the post-field prefill. The cheaper version, `field+selective@`K, recomputes only the field plus the K downstream tokens that carry most of the decision—the high-effect aggregator positions identified in Section 3, ranked by their causal effect on the decision—and reuses every other cached token unchanged. It is cheaper but *unreliable*: the smallest K that restores the decision is strongly model-dependent (below). *(ii) Append an erratum*: one salient line late in the context—"`[STATE UPDATE] field → new; overrides any earlier value and conclusion`"—so the decision token attends to a fresh, authoritative note (verbatim template in Appendix B). The erratum is the cheap, robust default: appended after the field (`field+erratum`) it matches the strong *hoist-to-end* oracle *without rewriting the prompt* (P(correct) 1.00 on the gated-task frontier, Figure 2a; Table 7), and being append-only it composes with prefix caching (Section 9).

Chain-of-thought, not model size, gates the cheapest edit. A third, near-free option—refresh the field's own KV alone (∼1% compute) and nothing else—works only when some later computation actually re-reads the field. A reasoning *chain* does exactly that, so with chain-of-thought the field-only edit alone recovers the decision (1.00 on Qwen3-8B); without the chain the *identical* edit on the *same* model is ignored (0.00) and the decision commits to the stale note (Figure 2c). The divider is therefore the CoT *mode*, not raw scale—reasoning-native models default to CoT, instruction-tuned ones to direct answers—so without CoT one of the two real fixes above is required. The dependence is two-level, and it is worth stating precisely: CoT gates *whether* the field-only edit can work at all (the binary 0.00/1.00 split above is set by mode, not size), while *conditional on* CoT the residual *stickiness*—how completely the cheap edit recovers the decision—is model-dependent (Figure 5a; and E3 of Section 7, where it climbs with scale within the Qwen3

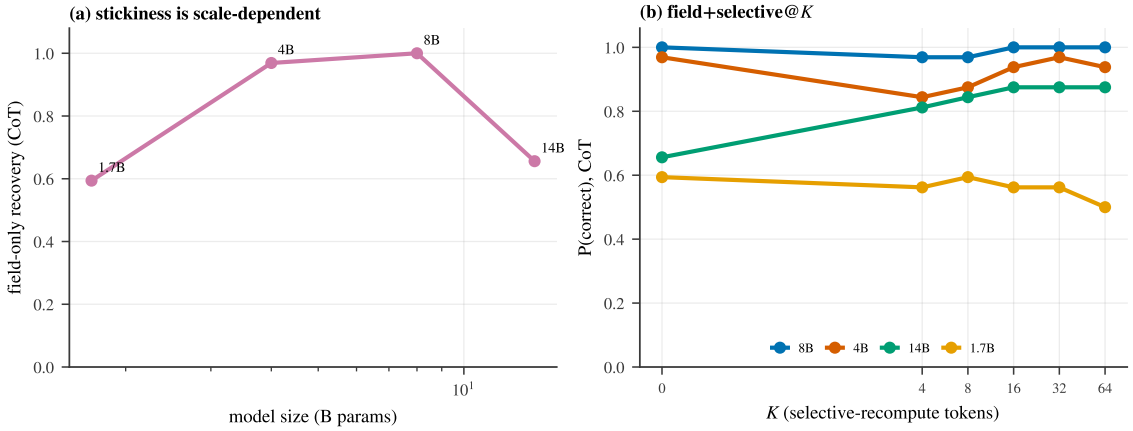


Figure 5: **Editing is model-dependent** (the editing landscape and the chain-of-thought split are previewed in Figure 2). (a) *Given* chain-of-thought (which gates whether the edit works at all), the residual stickiness is model-dependent: the near-free field-only recovery varies non-monotonically across model sizes. (b) `field+selective@`K: the minimal recompute $K^\star$ to reach full quality is likewise model-dependent.

family). These two dependencies—CoT-gating and per-model stickiness—are the whole story for the cheap edit, and we keep the rest brief here; the per-model stickiness, the `field+selective@`K sweep ($K^\star \approx 4$ at 8B but > 64 at 4B, varying non-monotonically with scale; Figure 5a,b), and the layer-wise account are detailed in Appendices C and F. We report `field+selective@`K honestly as a genuine but *unreliable* surgical tool, not a default.

Which edit to use: no single dominant method. A controlled comparison (full table in Appendix F) shows no single dominant method. Hoist-to-end is cheapest but demands prompt surgery and pre-identification of every field; `field+erratum` matches it with no surgery at a one-line append; `in_place` is near-free but only with chain-of-thought; a KV-deviation-ranked selective recompute (CacheBlend-style [38]) underperforms here because it chases changed keys rather than the tokens that *memoized the conclusion*. The practical recommendation is `field+erratum` as the robust default, with `in_place` as a free fast-path under chain-of-thought.

Why not edit the weights? A natural objection: to act on a changed field, why not edit the model's *weights* (ROME/MEMIT [25, 26]) or fine-tune, rather than the cache? We compare on the paper's gated task (status `pending`→`shipped`, so *cancel*→*deny*) against a faithful rank-one ROME—validated on the canonical factual edit first ("the Eiffel Tower is in" *Paris*→*Rome*, locality intact), so the baseline is not crippled—and a LoRA fine-tune [12] (Table 1; methodology in Appendix J). All of `field+erratum`, ROME, and LoRA *succeed* at flipping the target decision. But a weight edit is *global*: the same model instance can no longer hold `status=shipped` for one request and `pending` for another, so *all* concurrent orders that are genuinely still pending are wrongly flipped (cross-request contamination 1.0), and half of an unrelated decision battery drifts (0.5)—the ROME/fine-tune specificity tax—at 3–6 s per edit (plus a one-time covariance pass for ROME). The append-only erratum lives in a *per-sequence* cache: zero cross-request contamination, zero collateral, 114 ms, and it composes with prefix caching (Section 9). Weight editing targets *durable, global facts*; mutable per-request, per-turn state is the wrong job for it—which is precisely the niche the editable KV cache fills.

Table 1: **KV editing vs. weight editing** for mutable per-request state (Llama-3.1-8B). Weight edits succeed at the target yet are global (contaminate concurrent requests), damage unrelated decisions, and are 30–50× slower per edit.

method	flips decision?	edit latency	cross-req. contamination	collateral
`field+erratum` (KV)	✓	114 ms	**0**	**0**
`in_place` (KV)	✗ (no CoT)	71 ms	0	0
ROME (rank-one)	✓	5.6 s + 11.6 s cov	1.0	0.5
LoRA fine-tune	✓	3.1 s	1.0	0.5

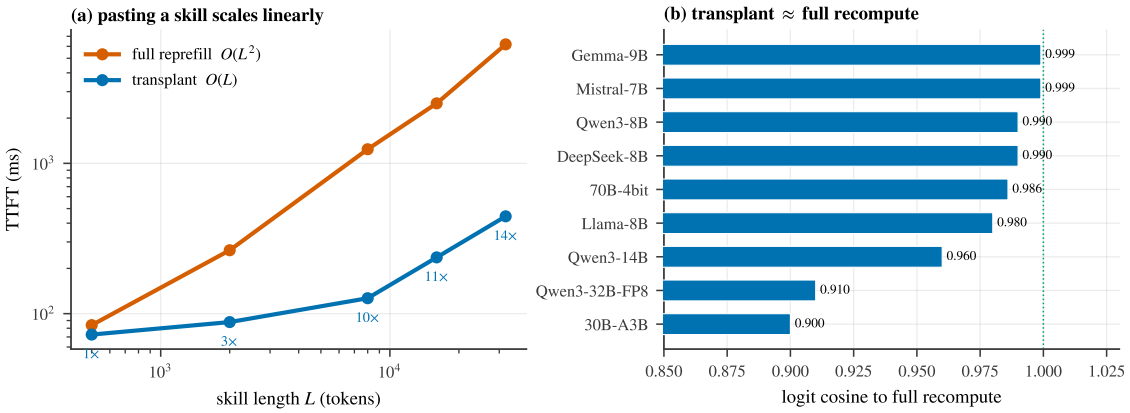


Figure 6: **Composing the cache.** (a) Pasting a precompiled skill is $O(L)$ vs. full reprefill's $O(L^2)$; TTFT speedup reaches 13.9× at 32k tokens. (b) The transplanted skill matches full recompute in next-token logits (cosine 0.90–0.999) across the model family.

5 Consequence II: the cache is composable

From mechanism to prediction. If a region's notes are localized and re-derivable from context the decision can still see, then the notes should be *position-portable*: we can compute a reusable chunk's KV once, in isolation, move it to a new absolute position, and splice it in. Concretely we precompile a long *skill* (a policy or tool specification), and because the attention library caches post-RoPE keys, we re-rotate the chunk's keys from their source positions to the target positions (values are position-free) before concatenating (Figure 4b). This is the position-aware reuse of Liu et al. [21]; our point is that the *mechanism predicts the splice should match a full reprefill*—the same next-token logits and decisions.

Transplant is behaviorally indistinguishable from full recompute. It does. The spliced skill matches a full reprefill in next-token logits with cosine 0.90–0.999 across the full model family (twelve models; full roster in Appendix A)—Qwen3-1.7B through 32B (including FP8 and the 30B-A3B Mixture-of-Experts), Gemma-2/3, Mistral-7B, Llama-3.1-8B and 70B, and DeepSeek-R1-Distill-Llama-8B (Figure 6b)—and on the models competent at the task it preserves *correct* skill-following across 8 diverse domains and 3 families (24/24, cosine 0.98–0.999), including 16/16 under chain-of-thought.

Context-robustness and the seam. A chunk precompiled in isolation matches one that attended to the real preceding context, because the decision re-derives from context it still sees. The one

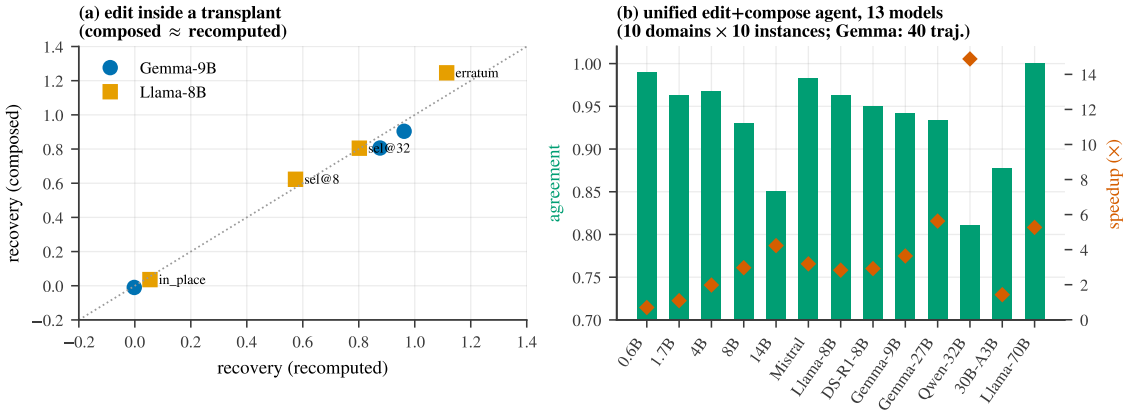


Figure 7: **Edit and compose can be combined.** (a) Editing inside a transplant: a field edited *inside* a transplanted skill reproduces the editing mechanism, and the *composed* cache matches the *recomputed* cache for every method (points on the diagonal). (b) A unified edit+compose agent over thirteen models (10 domains × 10 instances, 300 decisions; 120 on the two Gemma models): unified-vs-full agreement (bars) and cumulative-TTFT speedup (markers).

residual error is a *seam* at the chunk's start—the first tokens that, in a full prefill, would have attended to the now-missing prefix. Recomputing a few boundary tokens (*seam-repair*) closes it; this is exactly the boundary recompute of CacheBlend/EPIC/MPIC, and Section 3 explains why it is the boundary, specifically, that needs repair.

Linear-time TTFT. Full reprefill of a length-L skill is $O(L^2)$; transplant is $O(L)$ (a re-rotation pass plus the suffix). Time-to-first-token speedups grow with skill length: 3× at 2k tokens, 9.8× at 8k, and 13.9× at 32k on an 8B model (Figure 6a). A library of skills composes (decisions preserved for $N = 1$–4 concurrent skills).

Generality: content type, insertion position, and agentic tool-calling. Transplantation is not specific to rule-like skills. It preserves decisions for *facts/RAG* passages as well as rules; for chunks inserted in the system area *and* mid-trajectory as tool results; and—measured with actual function calls rather than a proxy—it preserves *agentic tool-calling* (N=108 with bootstrap CIs: function-call accuracy 1.00 on Mistral-7B, Llama-3.1-8B/70B, and Qwen3-8B; 0.97 [0.94, 1.0] on Qwen3-32B-FP8 and 30B-A3B, whose transplanted-vs-full tool-call *agreement* is 1.00). The one consistent exception is sliding-window attention (Gemma), which we diagnose and fix in Section 8. Full per-domain scorecards are in Appendix G.

6 Combining edit and compose

The keystone: editing inside a transplant. If editing and composing are truly two operations on the same object, then editing a field that lives *inside a transplanted skill* should behave exactly as editing a field in a normally-prefilled context. We test this directly: transplant a skill whose body contains a mutable field, then apply each editing method to that field and measure recovery, comparing the *composed* cache (skill spliced in) against a fully *recomputed* cache. The editing mechanism reproduces verbatim (Figure 7a): the in-place edit is weak (≈ 0.05), selective recompute recovers (sel@32 ≈ 0.80), the erratum is strongest, and crucially *composed* ≈ *recomputed* for every method (points lie on the diagonal) across Gemma-2-9B and Llama-3.1-8B. The notes a transplant pastes in are the same notes an edit amends—one notebook.

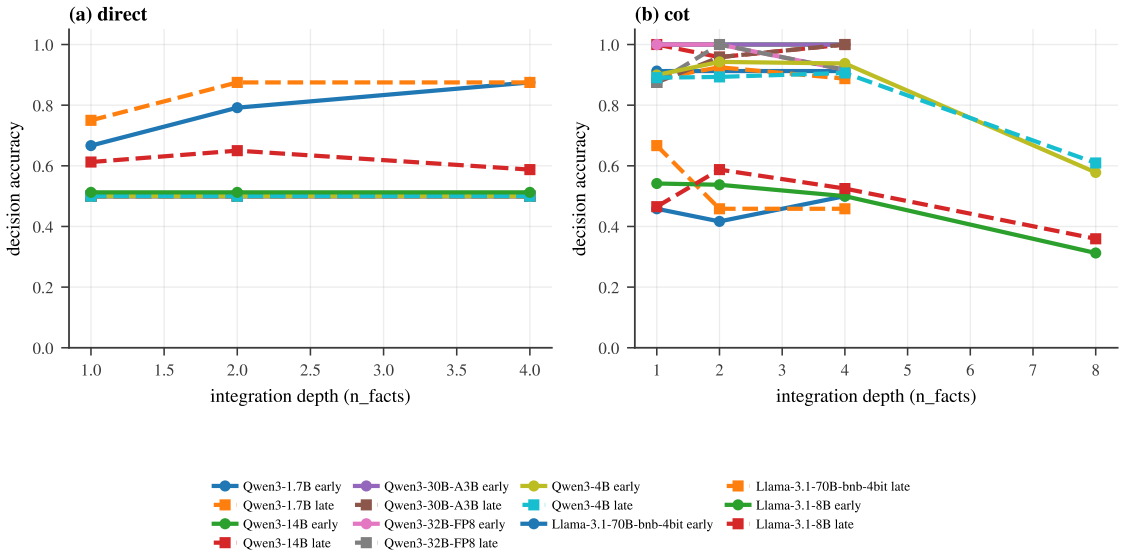


Figure 8: **E1 — placement and the pre-digestion cost.** Decision accuracy vs. integration depth (`n_facts`) for memory read *early* (solid) vs. *late* (dashed), across all evaluated models, under (a) direct answering and (b) chain-of-thought. Direct answering is at chance for the reasoning-native Qwen3 models (so CoT is the operative regime); under CoT, late placement forgoes prefill-time pre-digestion and carries a small accuracy cost vs. early that the text quantifies on Qwen3-4B (GEE-logistic, growing with memory length). Late placement is still preferred because it enables $O(L)$ editing/transplant (efficiency quantified in E5).

A unified edit+compose agent. We embody both operations in a single live agent loop: a long policy is *composed* once and never re-prefilled; as the world changes across turns the mutable state is *edited* by appended errata; and each turn reuses the longest cached prefix and prefills only the delta. Against a reprefill-every-turn baseline, over 10 agent domains $\times$ 10 instances (300 decisions) per model (40 instances, 120 decisions, on the two Gemma models) and across thirteen models (the twelve transplant models of Figure 6 plus Qwen3-0.6B; Appendix A), the unified path is decision-identical to full recompute with agreement 0.81–1.00 (e.g. Llama-3.1-8B 0.963, Mistral-7B 0.983, Llama-3.1-70B 1.00) at cumulative-TTFT speedups up to 14.9$\times$ (Figure 7b); the speedup scales with policy length $\times$ turns. Editing and composing therefore run in one loop, decision-identical to full recompute and at lower cumulative TTFT, across the model family. The leave-stale+erratum cache also does *not* compound error over long trajectories: across a 28-turn stress test where the gating field toggles every turn, the decision logits stay faithful to a full reprefill (cosine 0.99+, flat with trajectory length) with no systematic drift (Appendix H).

7 Application: editable and composable user memory

A concrete, high-value instance of this editable, composable notebook is *user memory*: the large, dynamically summarized profile of facts an assistant re-reads every turn. Memory is big (thousands–tens of thousands of tokens), reused across turns, and *mutated mid-session* by tool calls—so where it lives in the prompt is a dilemma the mechanism of Section 3 explains. Placed at the *front* (*[sys][MEM][traj]*), the trajectory memoizes memory-conditioned conclusions downstream, so a memory change forces a costly downstream reprefill (and an in-place edit is ignored). Placed at the

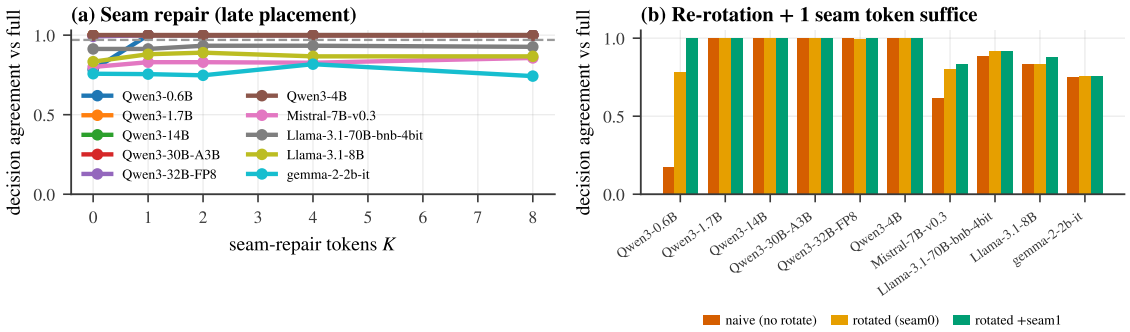


Figure 9: **E2 — memory transplant is faithful, to 70B.** (a) Decision agreement with full recompute under late placement; a single seam-repair token closes the start-of-chunk gap across ten models. (b) The re-rotation is necessary—the naive no-rotate control collapses, while rotated+1 seam token matches full recompute.

end (*[sys][traj][MEM]*), memory's KV depends on the whole trajectory and must be re-attended every turn. We resolve this by treating memory as a *skill that is also edited*: precompile it once in isolation, place it late (*[sys][traj][MEM][query]* so the decode reads it directly), RoPE-reposition it each turn, repair one boundary token, and edit it in place when it changes.

Placement and the pre-digestion cost (E1; Figure 8). Pre-digestion is real, and we quantify it. Under full recompute and chain-of-thought, reading memory *late* carries a small but *statistically significant* accuracy cost vs. *early* (Qwen3-4B, 192 personas per cell: GEE-logistic $\beta_{\text{late}} = -0.44$, $p = 0.018$), and the cost *grows with memory length*: the early−late gap is ≈0 at 2k tokens but +0.09 at 16k and +0.16 at 32k. The mechanism explains it—late placement forgoes prefill-time pre-digestion and relies on the decode/CoT to integrate raw memory. This is a *tradeoff, not free*: late placement is still preferred because it enables $O(L)$ editing/transplant at 2.3–4.3× lower TTFT and the transplant is faithful at a fixed placement (both shown below), so the net is small accuracy for large efficiency—with early placement preferable when memory is very long and accuracy-critical. (Direct one-shot decisions are at chance for the reasoning-native Qwen3 family *to 32B*, breaking only at Llama-3.1-70B, direct 0.81; CoT is the operative regime across scale—see Appendix I.)

Memory transplant is faithful, to 70B (E2; Figure 9). Across ten models from 0.6B to 70B, a precompiled+repositioned memory chunk reproduces the full-recompute decision logits with cosine 0.94–0.9996; a *single* seam-repair token closes the start-of-chunk boundary gap (Llama-3.1-8B late: cos 0.94 → 0.994). The cleanest decision-governance test is Llama-3.1-70B, whose decisions *genuinely vary* (so agreement is non-trivial, unlike the constant-answer regime of smaller models): the late, seam-repaired transplant reproduces the full-recompute decision 0.93 of the time at logit cosine 0.997, and *late placement beats early* (0.93 vs. 0.83) exactly as the mechanism predicts (the decode reads memory directly rather than through pre-digested notes across the transplant boundary). The no-rotation control collapses (late decision agreement 0.18 vs. 0.78 rotated), confirming the re-rotation is necessary.

Memory is editable mid-session (E3; Figure 10). When a stored fact is toggled, reusing the *stale* memory recovers the flipped decision essentially never (≤ 0.03). Every real edit recovers it, and—consistent with Section 4—under chain-of-thought the *near-free in-place edit* (one token recomputed) suffices, and *strengthens with scale*: correctness 0.92 → 0.99 → 1.00 across

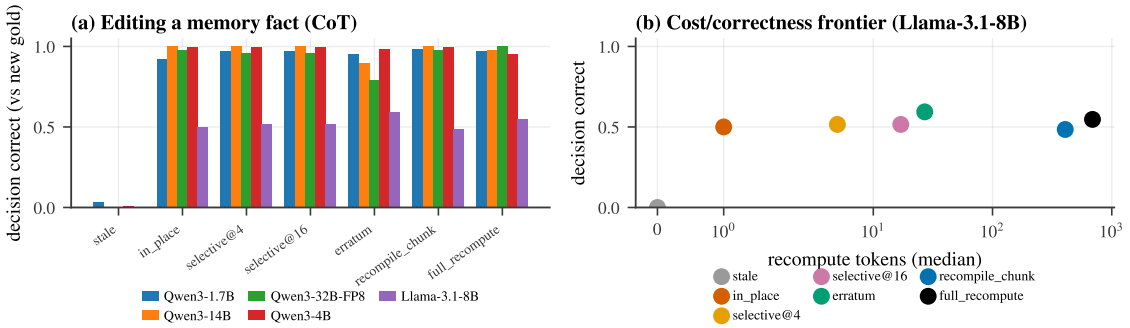


Figure 10: **E3 — memory is editable mid-session.** Reusing stale memory recovers a toggled fact essentially never; every real edit recovers it. Consistent with Section 4, the near-free in-place edit suffices under chain-of-thought and *strengthens with scale*, with the append-only erratum as the robust fallback.

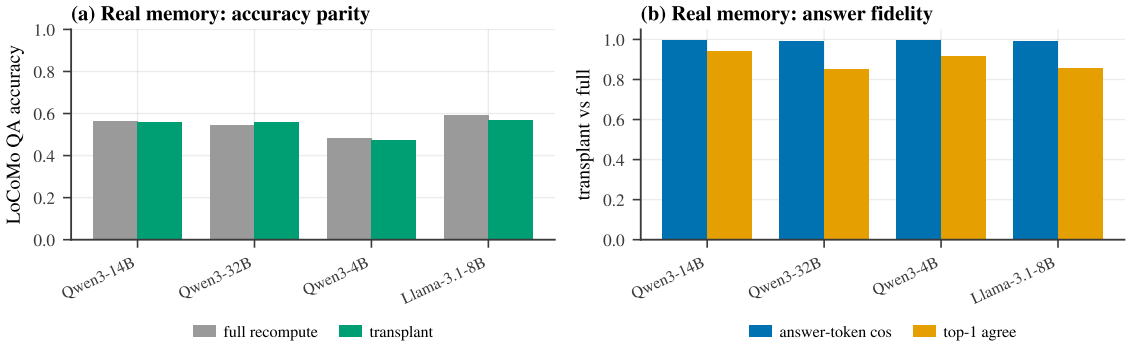


Figure 11: **LoCoMo — external validity on real conversations.** Over all 1,540 answerable questions per model, transplanting the multi-session dialogue memory is statistically equivalent to full recompute in QA accuracy (TOST) on the Qwen3 models and within 2.7 points on Llama-3.1-8B.

Qwen3-1.7B/4B/14B (and 0.98 at 32B). On models where the chain does not re-read the field, the append-only ERRATUM is the robust fallback (McNemar ERRATUM>`in_place` on Llama-3.1-8B, $p = 0.031$). This is the editing axis that concurrent KV-cache memory systems lack: MemArt [41] and EPIC [13] retrieve and splice *static* memory blocks position-independently but have no in-place memory *update*; our additions are the editing operation, the decision-governance lens, and the mechanism (Section 3) that explains why boundary recompute suffices.

Editing inside transplanted memory reproduces the mechanism. Editing a field *inside a transplanted memory chunk* in direct mode (Llama-3.1-70B, whose decisions vary so recovery is measurable) reproduces Section 3's memoization verbatim: refreshing the field's KV alone recovers only 0.55 of the flipped decision (the conclusion was memoized *downstream*, not in the field), and recovery climbs monotonically as more downstream tokens are recomputed ($0.55 \rightarrow 0.84$ at $K{=}16 \rightarrow 0.94$ full), exactly as for a normally-prefilled context. Under chain-of-thought this stickiness *dissolves*—a selective@K sweep is flat at $\approx$0.98 for all K (the chain re-reads the field, so $K^{\star}{\approx}1$). Edit and compose thus act on the same notes, and the direct/CoT dissociation matches the editing law of Section 4.

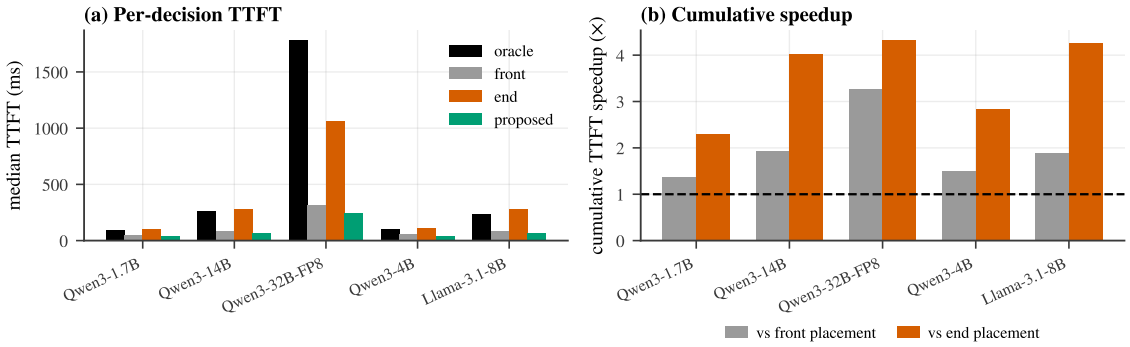


Figure 12: **End-to-end agent.** (a) Per-decision median TTFT for the proposed compose+edit agent vs. oracle, front, and end-placement baselines. (b) Cumulative TTFT speedup of 2.3–4.3× over reprefill-every-turn, growing with model size, at faithful next-token decisions.

Real memory: LoCoMo external validity (Figure 11). The synthetic gated decisions isolate the mechanism; to test real memory we run the long-conversation QA benchmark LoCoMo [24] (MemArt's setting): the multi-session dialogue (median ∼19.7k tokens) is the memory, precompiled and spliced before each question. Over *all* 1,540 answerable questions per model, transplant is *statistically equivalent* to full recompute in QA accuracy on Qwen3-4B, 14B, and 32B (TOST, margin 0.03: $|\Delta| \leq 0.015$) and within a small -2.7 points on Llama-3.1-8B, with answer-token logit cosine 0.991–0.998 throughout. The *compose* axis thus holds on real conversational memory, not only synthetic decisions.

Granularity is a free knob (E4; Figure 25). Splitting memory into S independently-precompiled blocks makes a localized edit $S\times$ cheaper (recompute one block) and is *decision-lossless* up to $S{=}16$ (agreement flat at 1.00 on Qwen3-4B), because the independent facts are integrated at read time, not within memory. Logit cosine degrades with S (0.998 → 0.953) but *identically whether the gating facts are contiguous in one block or split across blocks*—splitting *independent* relevant facts across independently-precompiled blocks does not specifically hurt. Genuinely cross-*referential* facts, however, do: in a controlled test (Appendix H) where a decision needs a two-hop chain (a `DEFINITION` line names which setting gates, whose value lives elsewhere), splitting the linked pair across a block boundary drops decision agreement with full recompute to 0.46 vs. 0.76 when the pair stays in one block—a 0.30 penalty (Llama-3.1-8B, $n{=}80$, full-recompute accuracy 0.85; McNemar $p{<}10^{-6}$), because block B's isolated precompute never attended to its referent A. The same split costs an independent fact pair only 0.04. Practical guidance: keep cross-referential facts in one block (Figure 25).

End-to-end agent (Figure 12). We implement a live agent that composes memory once, re-rotates it each turn, and edits it on tool-driven changes. Over 12-turn sessions (16–120 sessions per model) with ≈2k-token memories, against a reprefill-every-turn-at-the-end baseline it cuts cumulative time-to-first-token by 2.3–4.3× (growing with model size, to 32B), and against front-placement reprefill-on-change by 1.4–3.3×, while reproducing the full-reprefill next-token logits at a token-matched oracle (cosine 0.97–0.99). One honest caveat: greedy chains-of-thought are sensitive to sub-percent logit differences, so the exact reasoning *chain* is not always reproduced (chain agreement 0.31–0.78) even though the next-token decision is faithful and CoT *accuracy* is comparable; the clean decision-governance equivalence is the short-context editing result above (Section 4-style,

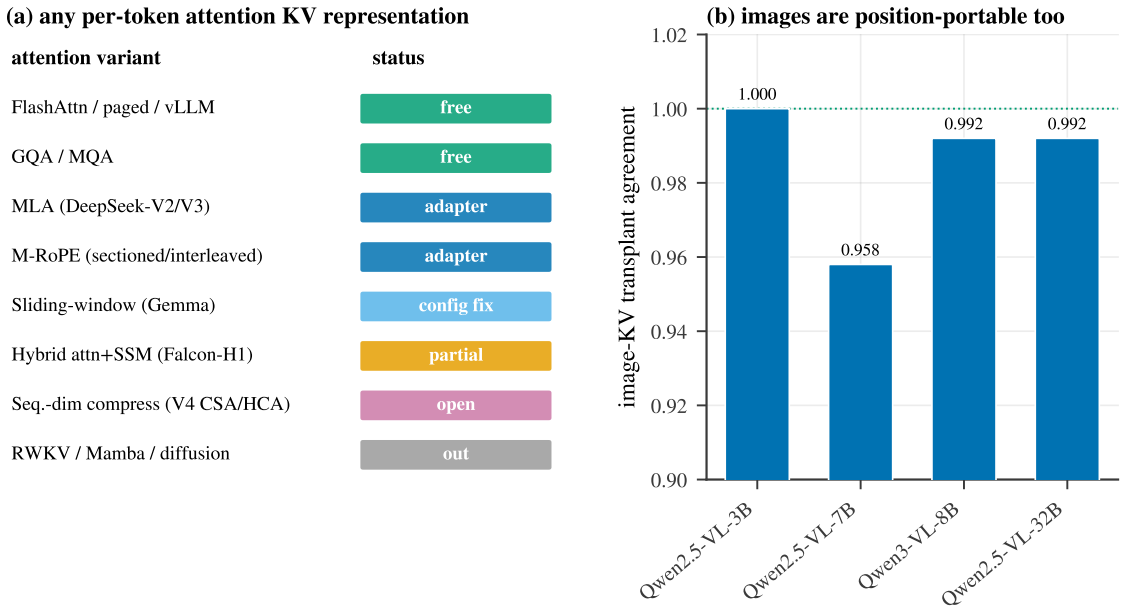


Figure 13: **Applicability to multimodal and new attention mechanisms.** (a) The operations work on any per-token attention KV representation; we map each attention variant as free / adapter / config fix / partial / open / out-of-scope. (b) Image-KV transplant is near-lossless across vision-language models—images are position-portable too.

agreement 0.89–0.95). User memory is thus a second instance of one set of notes that is both editable and composable.

8 Applicability to multimodal and new attention mechanisms

Scale, quantization, MoE. The transplant is faithful from 0.6B to 32B, on FP8 checkpoints, on a 30B-A3B Mixture-of-Experts, and on a 4-bit 70B model (feasibility 8/8, logit cosine 0.986); the unified agent of Section 6 likewise spans all thirteen.

Multimodal: images take notes too. In an agent trajectory an image costs a full prefill—the vision tower *plus* prefilling its >1k soft-tokens through the LM. Because image notes are also position-portable, we cache an image's LM-side KV once and splice it, re-running only text. Across 120 diverse VQA tasks per model (perception/reasoning/agentic), the spliced image is near-lossless versus full re-encode—agreement 0.958–1.0 on Qwen2.5-VL-3B/7B/32B and Qwen3-VL-8B (Figure 13b)—and reusing a cached image is 2.4–8.4× faster TTFT. Moving an image to a different trajectory position requires re-rotating only the temporal axis of M-RoPE; we handle both the *sectioned* (Qwen2.5-VL) and *interleaved* (Qwen3-VL) layouts, with position-shifted transplant lossless (agreement 0.99, Δ=161 positions). This subsumes MPIC's multimodal reuse [43], replacing its boundary-recompute with exact re-rotation.

The operations work on any per-token attention KV representation. They depend on the cache *representation*, not the attention kernel, so we map exactly where they hold (Figure 13a):

- **Free**—throughput optimizations that keep per-token KV: FlashAttention, paged attention/vLLM, and GQA/MQA (already used by every model we ran).
- **Adapter (implemented, validated)**—representation changes (diagrammed in Figure 23). *MLA*

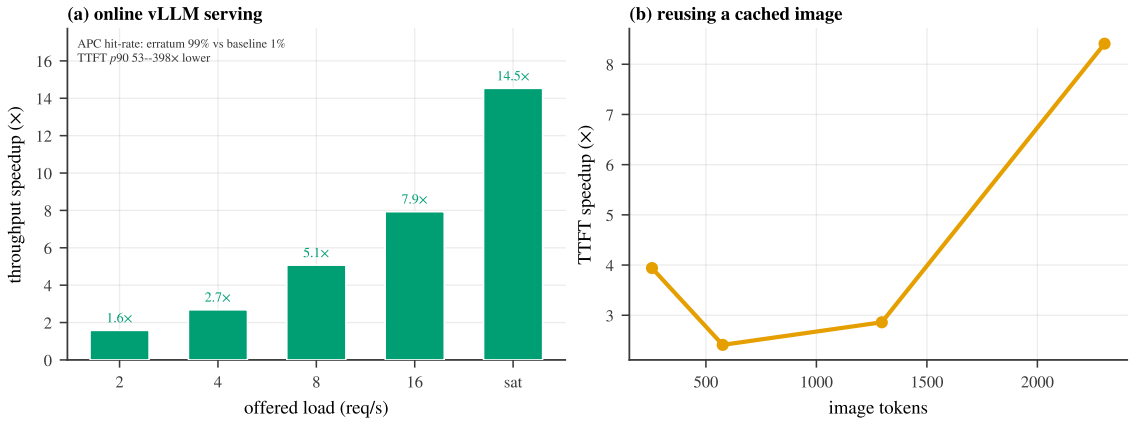


Figure 14: **Systems payoff.** (a) Online vLLM serving (V1 engine, CUDA graphs, continuous batching, APC, Poisson arrivals): the append-only erratum keeps the prefix cache-aligned (98.5% vs. 1% APC hit-rate), so its throughput advantage grows with offered load—up to 14.5× at saturation—while $p90$ TTFT is 53–398× lower. (b) Reusing a cached image (skipping the vision tower and image-token prefill) accelerates time-to-first-token, more so for larger images.

(DeepSeek-V2/Coder-V2) caches a position-free latent plus a small decoupled-RoPE sub-vector; our decoupled `k_pe` reposition re-rotates only that sub-vector, giving logit cosine 0.98 and composed-vs-full agreement 1.00 on DeepSeek-Coder-V2-Lite. *Interleaved M-RoPE* (above) is the other adapter.

- **Config fix**—sliding-window (Gemma): the default cache truncates sliding layers to the window, breaking uniform splices beyond it; keeping the *full* per-token KV and letting the attention *mask* enforce the window restores correctness (the previously-failing unified agent now runs at agreement 0.93–0.94; Figure 23c).
- **Partial**—hybrids with full-attention layers (Falcon-H1): the attention KV is transplantable but the per-layer Mamba scan-state is recurrent, not per-token, so a correct transplant must re-scan the Mamba path and saves only the attention fraction.
- **Open frontier**—sequence-dimension KV compression (DeepSeek-V4's CSA/HCA [6]) merges tokens into sub-token-count entries, so edit/splice become block-granular; DeepSeek-V3.2's sparse attention (DSA [5]) is MLA plus top-k selection and inherits our MLA adapter.
- **Out of scope**—no per-token attention KV: pure-recurrent (RWKV [27]), pure-SSM (Mamba [11]), and diffusion LMs. The prompt-level erratum still applies there, but it is not a KV operation.

9 Systems payoff

A real agentic environment. On the τ^2-bench retail environment [1]—single tool-decisions and a multi-turn autonomous-agent loop scored by the environment's own tool enforcement—an agent that reuses a stale cache after a state change fails the task—it acts on the outdated field value—while `field+erratum` preserves task success at a fraction of the recompute cost. On the real ∼1.4k-token retail policy, transplant reproduces the clean decision (composed = full on Llama-3.1 and Mistral); the one hard case, a long buried field that must *flip* a conclusion, requires the robust `field+erratum` edit rather than transplant-plus-bare- erratum—the same long-context lesson the

editing axis predicts, now in a real environment.

A comprehensive online serving benchmark. Because the erratum is append-only, it composes with standard automatic prefix caching (APC): the static prefix stays cache-aligned and only the short erratum (and the decode) is new work, whereas writing the new value *into* the prefix changes a cached block's content hash and invalidates every downstream block. We test this on vLLM's V1 engine as a real online server—`AsyncLLMEngine` with CUDA graphs, continuous batching, APC, and *Poisson* request arrivals at controlled offered load (not an offline batch)—over a shared ∼8k-token agent policy with one mutable field, measuring TTFT percentiles, throughput, and the engine's own APC hit-rate (Figure 14a; full configuration in Appendix J). The append-only edit keeps the prefix a cache hit (98.5% **vs.** 1.0% **APC hit-rate**); the in-prefix baseline is prefill-bound and *saturates at* ≈1.5 *req/s*, so its $p90$ time-to-first-token collapses under load (22–55 s) while the erratum stays 86 ms–1 s (53–398× lower TTFT). The throughput advantage *grows with offered load*—1.6× at 2 req/s up to **14.5**× at saturation—exactly as predicted for a compute-bound vs. cache-bound regime. The image-KV reuse of Section 8 contributes a complementary serving win—2.4–8.4× faster first-token as image size grows (Figure 14b)—by skipping the vision tower and image-token prefill entirely.

10 Limitations

Our operations require a per-token attention KV cache, so pure-recurrent, pure-SSM, and diffusion models are out of scope, and hybrid attention+SSM models are only partially served (the recurrent state is not transplantable; Section 8). The MLA and sliding-window adapters are validated but carry residual edge cases: a single transplanted *chunk that itself exceeds the sliding window* drops to logit cosine ≈ 0.89 (real skills are far smaller), and our small MLA checkpoints ship legacy-cache custom modeling that we shim. Sequence-dimension KV compression (DeepSeek-V4-class [6]) and cross-attention image caches are open: the unit of edit/transplant there becomes a compressed block rather than a token, which we have analyzed but not implemented. The `field+selective@`K surgical edit is unreliable (model- and domain-dependent stickiness) and we present it as such, not as a default. Finally, while several studies use synthetic policies for controlled measurement, we mitigate this with the real τ^2-bench retail policy and real images; broader real-workload evaluation remains future work. The neuron-level analysis (Appendix E) is narrower than the rest of the mechanism study: it covers one model in depth (Llama-3.1-8B, with Qwen3-8B replicating only attribution and patching), its finding that single-neuron ablations undercount the joint effect is borderline at 95%, and its labels and simulations come from a single LLM (MiMo-v2.6-pro). More broadly, our operations exploit a mechanism that arises *for free* in today's models; the direction this work points to is a KV cache *programmable* by design—models trained to expose composable, editable notes—which we leave to future work.

11 Conclusion

A surgical edit to a field's KV is ignored not because the cache is fragile but because the model has already done the work: at prefill it computes the field-conditioned *conclusion* and writes it onto downstream aggregator tokens, so the decision only reads those notes back. This reframes what a KV cache is—not a write-once byproduct of prefill, but a structured record of intermediate conclusions that we can read, amend, and rearrange. Two capabilities follow from the one mechanism: *editing* the notes (a late, salient erratum in place of recomputation) and *composing* them (repositioning and splicing a precompiled skill in $O(L)$ time). The mechanism also says where to intervene. The write is spread over many attention heads and tens of thousands of MLP neurons, whose labels read cleanly but do not predict what the neurons do. The result, by contrast, sits on a few cached

positions that a dozen heads read additively. The cache, not the weights, is the natural place to edit.

The question this opens is larger than either operation. If prefill routinely deposits reusable conclusions into the cache, then much of what a model has worked out mid-context is already written down somewhere we can inspect and overwrite. Editing and composition are two uses of that record, and they work today even though no model was trained for them. The larger opportunity is to make this explicit: models trained to be *aware* that their KV can be composed and edited—exposing notes that are cleanly addressable, splice-able, and revisable by design—would turn the cache from a linear, append-only log into a *programmable* memory the system reads, writes, and rearranges. Once the KV cache is programmable rather than merely linear, context engineering changes shape significantly: *skills, memory, and state become first-class, reusable cache objects* rather than text re-prefilled on every turn. This paper is a first step toward the vision of programmable KV cache—a notebook the model keeps for itself, and one it can learn to read and edit.

Acknowledgements

We thank Junhao Hu, first author of EPIC (Efficient Position-Independent Caching) [13] and a co-author of CacheSlide [21], for discussions on editable KV cache that inspired this research project. This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow, in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Fable 5 and Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu, and William Saunders. Language models can explain neurons in language models, 2023. OpenAI, https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html.

[3] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[4] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[5] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[6] DeepSeek-AI. DeepSeek-V4 technical report, 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.

[7] Leo Gao. Neural networks are not already clean. LessWrong, October 2026. URL https://www.lesswrong.com/posts/Dkh7pXKjoNWkkxQDz/neural-networks-are-not-already-clean. Posted 17 October 2026.

[8] Gemma Team. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[9] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

[10] Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[14] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, et al. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[15] Albert Q. Jiang et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

[16] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, et al. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.

[17] János Kramár, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing LLM behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.

[18] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[19] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, et al. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[20] Jack Lindsey et al. On the biology of a large language model. *Transformer Circuits Thread, Anthropic*, 2025.

[21] Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, 2026.

[22] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of ACM SIGCOMM*, 2024.

[23] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, et al. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.

[25] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[26] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations (ICLR)*, 2023.

[27] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, et al. RWKV: Reinventing RNNs for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[28] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[29] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[31] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

[32] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.

[33] Eric Todd, Millicent Li, Arnab Sen Sharma, Aaron Mueller, Byron C Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[34] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, et al. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[35] Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations (ICLR)*, 2023.

[36] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[37] Jingbo Yang et al. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025.

[38] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2025.

[39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[40] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[41] Yuan Zeng, Pengfei Zuo, Min Lyu, Xingkun Yang, Huatao Wu, Yinlong Xu, and Zhou Yu. KVCache-centric memory for LLM agents, 2025. Submitted to ICLR 2026; OpenReview, 18 September 2025.

[42] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[43] Shiju Zhao, Junhao Hu, Rongxiao Huang, Jiaqi Zheng, and Guihai Chen. MPIC: Position-independent multimodal context caching system for efficient MLLM serving. *arXiv preprint arXiv:2502.01960*, 2025.

Table 2: Model zoo. "role" indicates the experiments a model appears in.

model	family / attention	precision	role
Qwen3-0.6B/1.7B/4B/8B/14B	Qwen3 (GQA)	bf16	mechanism, edit, compose, agent
Qwen3-32B	Qwen3 (GQA)	FP8	compose, agent, scale
Qwen3-30B-A3B	Qwen3 MoE (GQA)	bf16	compose, agent, MoE
Llama-3.1-8B / 70B	Llama (GQA)	bf16 / 4-bit	mechanism, edit, compose, agent
Mistral-7B	Mistral (GQA)	bf16	mechanism, compose, agent
Gemma-2-9B / Gemma-3-27B	Gemma (sliding-window)	bf16	mechanism, compose, agent, sliding-window fix
DeepSeek-R1-Distill-Llama-8B	Llama (GQA), reasoning	bf16	edit, compose, agent
DeepSeek-V2-Lite / Coder-V2-Lite	DeepSeek (MLA)	bf16	MLA adapter
Falcon-H1, Falcon-Mamba	hybrid / pure SSM	bf16	architecture boundary
Qwen2.5-VL-3B/7B/32B, Qwen3-VL-8B	VL (M-RoPE)	bf16	multimodal image-KV

A Models evaluated

Table 2 lists the models used across the paper, spanning the families Qwen3 [29], Llama-3.1 [10], Gemma-2/3 [8], Mistral-7B [15], and DeepSeek-V2/Coder-V2 [3]. All runs are on a single RTX PRO 6000 (Blackwell, 96 GB); FP8 and 4-bit checkpoints are the official quantized releases.

B Worked examples: prompts, errata, and responses

The gated decision (the paper's running example). The mechanism and editing experiments use prompts of the following shape (scenario `account_role`, the verbatim template from the released harness; the thirty neutral filler rules are abridged and long lines re-wrapped for display). The prompt is wrapped in each model's chat template and the decision is read at the final position:

```
You are a customer-support agent for an online retailer.

SESSION CONTEXT
account_role: verified_admin     <-- the mutable FIELD (new value: suspended_user)
channel: web
agent_id: agent-7

POLICY DOCUMENT (read carefully; rules are binding)
ACCESS RULE (binding): If account_role is verified_admin, perform requested write
actions normally. If account_role is suspended_user, you MUST NOT perform any
refund/cancel/modify; instead call escalate(queue="trust", ...) and nothing else.
R1. Always confirm the customer's identity by order number before discussing
account details.
[... R2-R30: twenty-nine more neutral filler rules ...]

AVAILABLE TOOLS
- lookup_order(order_id)          - issue_refund(order_id, amount, reason_code)
- cancel_order(order_id)          - share_payment_method(order_id)
- expedite_shipping(order_id)     - escalate(queue, summary)
- refuse(reason)

CONVERSATION SO FAR
user: Please refund 40.00 to my order A4471 for the shipping delay.
assistant: Let me check the account and policy before acting.

TASK
Decide the single next tool call. Respond with exactly one line:
tool_call: <name>(<args>)
Next action:
```

The four cache states of Section 3 are built from this prompt: *stale* prefills the old value, *oracle* the new, *field-only* overwrites the KV of the `account_role` value span (two tokens of a ∼958-token templated prompt, 0.2% recompute), and *full-downstream* recomputes everything after it. In the position-resolved causal patch (Figure 3b,c), the highest-effect downstream positions for a representative instance sit not on the field span but on aggregator/delimiter tokens—the end-of-rule punctuation (around "`...nothing else.`") and the TASK-block instruction tokens (around "`tool_call: <`") that the decision attends through.

The erratum, verbatim. The editing fix appends one salient line ahead of the decision prompt (the `field+erratum` arm of Section 4):

```
[STATE UPDATE] account_role has just changed to suspended_user; this overrides
any earlier value. Apply the current value.
```

The wording ablation of Figure 3d varies this line from a bare restatement of the value to the aggressive "disregard your earlier conclusion and re-evaluate", which underperforms.

Recorded responses (Qwen3-8B, reasoning mode; one released instance). From the record `thinking_qwen3_8b_think.json` (in `results/`), abridged to the answer head:

```
oracle (clean prefill of suspended_user), 249 thinking tokens:
  tool_call: escalate(queue="trust", summary="Request to refund $40.00...")
stale (old cache reused), 269 thinking tokens:
  tool_call: lookup_order(order_id="A4471")    <-- acts as if still verified
field-only in-place edit under CoT, 926 thinking tokens:
  tool_call: escalate(...)                     <-- chain re-reads the field
```

The in-place edit recovers the oracle decision under reasoning, but the recomputation has moved from prefill into the chain (926 vs. 249 thinking tokens, ∼3.7×); without reasoning, the same edit is simply ignored (Figure 3a).

The dissociation pair (Appendix C). The conclusion/content control holds the field byte-identical and flips one trigger token inside the rule:

```
ACCESS RULE (binding): If account_role is {trigger}, you MUST NOT perform any
refund, cancel, or modify action and must instead call escalate(queue="trust",
summary=...). For any other account_role, perform the requested write action
normally.
```

With the field fixed at `verified_admin`, setting `trigger=verified_admin` makes escalation correct, while `trigger=suspended_user` (field unchanged) makes the write action correct: the two prompts differ in exactly one token, the field is constant, and only the *conclusion* flips.

A transplanted skill (Section 5). A representative precompiled skill (the 20 neutral guidelines abridged):

```
# SKILL: REFUND_POLICY
You handle refund requests. Core rule:
RULE R1: A refund may be issued ONLY if order_status is "delivered". For any
other status (pending, shipped, cancelled, returned) you MUST refuse the refund
and escalate to a human.
- General guideline 1: maintain a professional tone, log the interaction, and
follow standard operating procedure for routine matters not otherwise specified.
[... general guidelines 2-20 ...]
End of REFUND_POLICY skill.
```

The skill is prefilled once in isolation (positions $0..L-1$), its keys re-rotated to the target offset, and spliced after the system prompt; the task suffix

```
Order #7731 has order_status = "pending". The customer requests a refund. Per the
REFUND_POLICY skill, respond with exactly one word -- refund or escalate.
Decision:
```

then reads the spliced notes, and the decision (`escalate`) matches a full reprefill (Figure 6).

C Deep mechanism controls: dissociation, timing, specificity, writability

These four controls (and an off-template generalization) tighten Section 3 from *decodability* to *causation, timing, and write-access*. Each runs on three models (Qwen3-8B, Qwen3-4B, Llama-3.1-8B); scripts `esys/mechd_*.py`, records `results/mechd_*`.

(i) Dissociation (Table 4, left). A polarity-parameterized rule names a single *trigger* value that selects the safe action, so flipping the trigger inverts the conclusion while the field value is byte-identical across the pair (the two prompts differ in exactly one token, inside the rule). Transplanting the post-trigger *notes* from the opposite-conclusion cache carries essentially the entire flipped conclusion, whereas patching the differing rule token carries none—with the field held constant, the decision is reading a memoized conclusion, not field content. A logit-probe finds both the conclusion and the field identity linearly decodable from the same downstream delimiter, so decodability cannot itself adjudicate; the causal transplant is required.

(ii) Timing. Using `output_hidden_states` on the prefill, the conclusion becomes linearly decodable (group-CV logistic probe, conclusion $\perp$ field by the 2×2 design) on the downstream aggregator at layer-depth $0.39/0.39/0.31$ (Qwen3-8B/4B, Llama-3.1-8B), while the decision token's logit-lens margin reaches its final sign only at depth $0.75/0.77/0.73$—the note is written $\sim$12 layers before it is read, within one prefill.

(iii) Specificity (Table 4, right). Ranking downstream positions by individual transplant effect, the top-8 recover 0.74–0.79 of the decision; 8 random downstream positions recover $\leq$0.035. The conclusion sits on a few specific aggregator/delimiter tokens, not diffusely.

(iv) Writability. Overwriting an otherwise-consistent cache's downstream notes with the *opposite* conclusion's notes (the field token and prefix left intact and still implying the original answer) drives the decision to the injected conclusion: continuous recovery $0.99/0.98/1.02$, with the top-8 note positions already flipping the belief in most instances. Editing (Section 4) is the benign use of this same write-access.

Off-template generalization. Re-running field-only vs. full-downstream recovery on (a) a 2-hop lookup, (b) free-form conversational phrasing, and (c) attribute lookup: field-only recovery is $\approx$0 for (a) and (b) across models (multi-hop -0.012 to $+0.006$; natural -0.012 to $+0.054$) with full-downstream 1.00, confirming the mechanism is not a template artifact; for near-verbatim attribute lookup (c) the field is partly a copy and carries 0.25–0.63, bounding the claim to *derived* conclusions.

Cross-family replication (Gemma-2, Mistral). To rule out a Qwen3/Llama tokenizer artifact we re-ran five probes (the locality probe and the four controls above) on **Gemma-2-9B** and **Mistral-7B** with a tokenizer-robust readout (space-prefixed single-token `cancel`/`deny`; Gemma-2's soft-capping kept intact, as the attention-knockout hook used only by the original circuit-knockout probe otherwise corrupts it), on the gated cancel/deny task (n=18 primary, 18 dissociation each; Table 3). All five replicate: field-only ≈ 0 vs. full-downstream 1.0; dissociation trigger-only ≈ 0 vs. notes $\approx$ 1.0; top-8 $\gg$ random-8; injection ≈ 1.0; and write-before-read timing. Two honest notes: Mistral's field-only recovery (0.137) is slightly above zero (still far below full-downstream 1.0), and random-8 specificity runs higher on this task (0.48–0.53 vs. $\approx$0.02 on the Qwen/Llama tool-call task) though

Table 3: Cross-family replication of the five deep-mechanism probes (Gemma-2-9B, Mistral-7B). Recovery toward the target conclusion; bootstrap means.

model	field-only	full-down	trigger-only	notes	top-8 / rand-8	write/commit depth
Gemma-2-9B	0.005	1.0	−0.00	1.0	0.96/0.48	0.26/0.48
Mistral-7B	0.137	1.0	0.001	0.995	0.94/0.53	0.19/0.47

Table 4: Deep mechanism controls (recovery toward the target conclusion; three models). Left: dissociation—patching the differing rule token vs. the downstream notes, field held identical. Right: specificity—top-k vs. random-k downstream positions, matched count.

(i) Dissociation			**(iii) Specificity**				
model	trigger only	notes	model	top-8	rand-8	top-16	rand-16
Qwen3-8B	−0.007	1.004	Qwen3-8B	0.78	0.009	0.92	0.06
Qwen3-4B	−0.007	1.009	Qwen3-4B	0.74	0.035	0.86	0.04
Llama-3.1-8B	+0.007	0.998	Llama-3.1-8B	0.79	0.005	0.86	0.04

top-8 (≈ 0.95) still dominates.

D A component-level circuit for memoized inference

Section 3 localizes the memoized conclusion (which tokens, which layers, a linear probe). This appendix pushes from *localization* to a *component-level* account—which attention heads, which direction, attention vs. MLP—with five interventions on the polarity 2×2 task, where the field is held *byte-identical* across the conclusion flip (only one rule-trigger token differs), so every "conclusion" signal we attribute cannot be field content. Primary model Llama-3.1-8B, replicated across *four families*—Qwen3-8B, Gemma-2-9B, Mistral-7B (Table 5)—all numbers are causal-intervention recoveries with bootstrap CIs over n=12 instances (Figure 15; scripts `esys/circ_*.py`, records `results/circ_*`).

(1) Named read and write heads (Figure 15a,f). We rank heads by direct attribution and confirm causally by patching a single head's output (clean↔corrupt) and re-reading the decision. *Read heads*—at the decision token—form a concentrated, nameable set: patching the top k jointly recovers 0.19/0.45/0.59/0.70/0.78 of the decision at k=1/3/5/8/12 on Llama-3.1-8B, and the top-12 reach 0.77/0.72/0.82 on Qwen3-8B/Gemma-2-9B/Mistral-7B (random-head control ≈ 0 on all four). The strongest read heads sit in late layers and attend decision→aggregator (Llama `26.3` at 0.46, Qwen `26.25` at 0.43, Gemma `26.9` at 0.47, Mistral `20.21` at 0.31). *Write heads*—at the aggregator, at prefill—are more distributed: single-aggregator patching of the top heads saturates at 0.28/0.08/0.24/0.11 (Llama/Qwen/Gemma/Mistral), because the write is spread over *many* aggregator tokens (the suffix-concentration of Section 3), so patching one position under-counts it. The decision-side read is thus a tight bottleneck; the write is diffuse. Ablating heads jointly versus one at a time (Appendix E) sharpens both halves: the twelve read heads are additive (joint drop 0.49 vs. summed single drops 0.52) with no compensation among them, while the write heads are partly redundant (single-head sufficiencies sum to 2× their joint sufficiency).

(2) A causal conclusion direction (Figure 15b). A single difference-of-means direction $\hat{d}$ on the aggregator residual (fit *leave-one-scenario-out*, field-balanced) transfers the conclusion: injecting the $\hat{d}$-component of the clean−corrupt residual into a corrupt run recovers 0.22–0.23 of the

decision at Llama L12–14—$\approx$ 39% of the full single-site residual patch and $\sim$25$\times$ a random 1-D direction (0.01); the difference-of-means direction beats a logistic-probe direction (0.02), a known DAS phenomenon. So the conclusion has a real shared linear component, but it is *redundantly* coded (projecting $\hat{d}$ out of a clean run drops the decision only $\sim$0.17). The direction is causal but similarly redundant on Gemma-2-9B (along 0.10 vs. random 0.00, of full 0.61) and Mistral-7B (along 0.07 vs. random 0.00, of full 0.36). On Qwen3-8B the conclusion is committed so strongly (logit gap $\approx$22 vs. $\approx$4) that single-site directional recovery is near zero in relative terms—the read-side circuit, which acts at the decision bottleneck, is the robust cross-family result.

(3) A sparse SAE feature: decode $\neq$ cause (Figure 15e). We train a TopK SAE (16k dict, k=32, FVU $\approx$0.001) on layer-14 residuals over diverse prompts. The conclusion is *sparsely decodable*: two features reach AUC = 1.00 at separating SAFE from UNSAFE on the aggregator (field-controlled), with $\sim$5 features above 0.95. Yet it is *causally distributed*: clamping the single best feature to its SAFE level in a UNSAFE run recovers ≈ 0, while clamping the top $3/10/30$ jointly recovers $0.30/0.50/0.54$ (random-feature control ≈ 0; necessity 0.30–0.33). A feature can read the conclusion out perfectly without being causally sufficient alone—the feature-level echo of the decodability-vs-causation dissociation in Section 3.

(4) Causal scrubbing of write$\rightarrow$note$\rightarrow$read (Figure 15d). We test the hypothesis "the decision is a function of the conclusion carried by the aggregator note" by resampling activations from inputs that agree with that labelling (position-aligned KV transplants from same-scenario donors). *Faithfulness*: resampling the entire downstream from a *same-conclusion* different-input donor preserves the decision (drift 0.02–0.04 of the gap; decision-logit cosine 0.999–1.000) on both models. *Necessity*: resampling only the *note* from an *opposite-conclusion* donor flips the decision $0.79/0.68/0.82/0.71$ (Llama/Qwen/Gemma/Mistral), whereas resampling the rest of the downstream moves it only $0.19/0.32/0.18/0.30$. The note governs; everything else is interchangeable.

(5) Attention writes the note (Figure 15c). Decomposing the SAFE$-$UNSAFE aggregator residual into per-layer attention- and MLP-block contributions (exact: the embedding is identical across the pair), attention contributes the majority of the write at the causal conclusion layer—$0.62/0.60$ at Llama L12/L14, and a striking 0.82–0.92 on Gemma-2-9B—with Qwen3-8B (0.44–0.56) and Mistral-7B (0.42–0.66) closer to parity. Attention is the primary or co-primary writer that routes the field/rule information onto the aggregator in every family, with the MLPs contributing the remainder. A whole-block patch on Llama-3.1-8B (24 pairs; Appendix E) agrees: patching every attention output at the top-8 aggregators recovers 0.89 of the decision, every MLP output 0.39 (the two need not sum to 1, since the blocks interact).

Summary. The picture is a *distributed write, concentrated read*: at mid layers, attention routes the field-conditioned conclusion onto aggregator tokens (Exp. 5), in a redundant code that is sparsely decodable but causally spread over $\sim$10–30 features / a shared low-rank direction (Exp. 2–3) and over tens of thousands of MLP neurons (Appendix E); a small, nameable set of late read heads then funnels it into the decision logit (Exp. 1), and causal scrubbing confirms the note alone governs the decision (Exp. 4). This connects the "models take notes" account to the head-level circuits of Wang et al. [35] and to delimiter-token aggregation [20], while remaining a statement about the *KV cache* an inference system already stores.

Table 5: **The circuit replicates across four families.** Read = decision recovery from jointly patching the top-12 named read heads (control in parens). Write = top-k write-head recovery at the single top aggregator (distributed, so a floor). Scrub: faithfulness drift under same-conclusion resampling (want $\approx$0) and note-vs-rest interchange recovery. Attn = attention share of the write at the causal layer. Dir = 1-D conclusion-direction recovery (along $\hat{d}$ vs. random).

model	read (top-12)	write	scrub: drift / note / rest	attn share	dir: along / rand
Llama-3.1-8B	0.78 (0.00)	0.28	0.04 / 0.79 / 0.19	0.60–0.62	0.22 / 0.01
Qwen3-8B	0.77 (0.00)	0.08	0.04 / 0.68 / 0.32	0.44–0.56	— (over-committed)
Gemma-2-9B	0.72 (0.00)	0.24	0.02 / 0.82 / 0.18	0.82–0.92	0.10 / 0.00
Mistral-7B	0.82 (0.00)	0.11	0.03 / 0.71 / 0.30	0.42–0.66	0.07 / 0.00

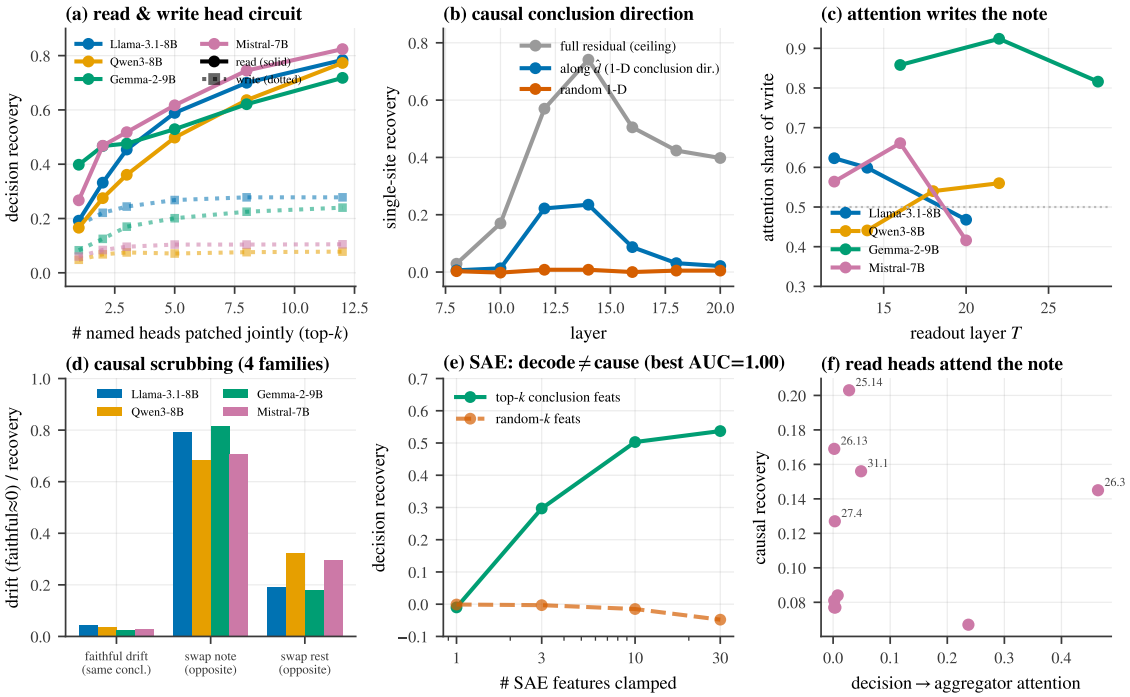


Figure 15: **A component-level circuit for memoized inference** (deep dives Llama-3.1-8B; head, write, and scrubbing panels show all four families). (a) Cumulative decision recovery from jointly patching the top-k named *read* heads (concentrated, $\sim$0.78) vs. *write* heads (distributed). (b) A leave-scenario-out difference-of-means *conclusion direction* causally transfers the decision far above a random 1-D direction. (c) At the causal conclusion layer, *attention* writes the majority of the note. (d) Causal scrubbing: same-conclusion resampling is faithful (drift$\approx$0); swapping the *note* to the opposite conclusion flips the decision while swapping the rest does not. (e) An SAE feature *decodes* the conclusion perfectly (AUC$=$ 1.0) yet is not causally sufficient alone—the cause is spread over $\sim$10–30 features. (f) Read heads with the largest causal effect are those that attend decision$\rightarrow$aggregator.

E Neuron-level anatomy of the note

Appendix D resolves the note to heads, a direction, and SAE features. This appendix goes one level finer, to individual MLP neurons, and asks three questions: who writes the note, whether a few neurons carry it, and whether neurons that *read* as clean actually *explain* it. Scripts, records, and a number-by-number map are in `followup/` (`followup/README.md`).

Setup. Llama-3.1-8B-Instruct (bf16) on the polarity task's full 2×2: 3 scenarios × 4 order ids × 2 field values = 24 SAFE/UNSAFE pairs, each with the field byte-identical and only the rule trigger flipped. The score is $S = \text{logit[safe]} - \text{logit[unsafe]}$ at the decision token, and every effect is normalized by $S_{\text{SAFE}} - S_{\text{UNSAFE}}$. The note positions are the top-8 aggregators ranked by KV-patch effect. A *neuron* is one SwiGLU intermediate unit (an input to the MLP down-projection): $32 \times 14{,}336 = 458{,}752$ of them. We rank neurons by attribution patching (AtP) [17], $(a_{\text{SAFE}} - a_{\text{UNSAFE}}) \cdot \partial S/\partial a$ at the note positions, a first-order estimate of each neuron's patch effect. Rankings use 18 pairs (3 order ids); every result is reported on all 24, and the held-out order id agrees (top-30 patch 0.29 vs. 0.30). The baseline reproduces Appendix C: top-8 transplant 0.793 (9 pairs; paper 0.79) vs. 0.017 for random-8, and field-only recovery −0.021 (paper −0.028). Qwen3-8B replicates the attribution and patching steps.

Attention writes; the MLP share is diffuse. Patching every attention output at the note positions recovers 0.89 of the decision; patching every MLP output recovers 0.39 (AtP puts attention's share at 0.78). Within the MLPs no small set carries the note (Figure 16a–b). The top 10 neurons hold 0.9% of the |AtP| mass, the top 30 hold 1.9%, and 50% needs 28,129 neurons (27,304 on Qwen3-8B). Patching the top 1/10/30/100/300 recovers 0.03/0.15/0.30/0.52/0.72 of the decision (Qwen3-8B: 0.03/0.16/0.25/0.39/0.55), while layer-matched random neurons stay within ±0.01. Signed contributions cancel: the top-300 patch (0.72) exceeds the all-neuron patch (0.39) because the remaining neurons push the other way. The 30-neuron result mirrors the SAE result of Appendix D (30 features recover 0.54). The MLP write peaks at layers 11–16 on Llama (L13 and L14 together hold ≈26% of |AtP|) and at L17–22 on Qwen3-8B (Figure 16c), the same mid-depth band where the note becomes decodable (Appendix C).

Activation sign. SwiGLU neurons take both signs, and the negative side matters in aggregate (Figure 16d). Zeroing every negative activation at the note positions removes 0.55 of the decision, zeroing every positive one removes 0.70, and zeroing both removes 0.92; negative activations also carry 0.36–0.49 of the |AtP| of the top 10 to top 1,000 neurons. But the negative side is not concentrated: the 8 most important negative-side neurons remove 0.011 (sign-matched random: 0.005), and the 8 most important positive-side ones remove 0.065.

Joint versus one-at-a-time ablation. For each carrier type we compare removing the whole set at once with the sum of removing each member alone (Table 6, Figure 16e). A ratio of joint to summed drop near 1 means the members act independently; above 1, single ablations undercount; below 1, members overlap. Decision logits are computed in fp32, and baselines use the same batch size as the ablated runs, since bf16 batch numerics otherwise add a +0.06 bias. The *read heads* are additive (0.95). Removing one does not make the others take over (compensation −0.04), and the remaining downstream heads absorb only 0.17 of an ablated head's direct effect. On the write side, single-head *sufficiencies* sum to 2× the joint sufficiency (ratio 0.50): several write heads each carry an overlapping copy of the note. For *MLP neurons* the opposite holds. One-at-a-time ablations capture only 0.53 of the joint effect of the top 30 (0.097 vs. 0.181 of the decision), so per-neuron audits understate what neurons do by about half. This is not compensation among the measured neurons, whose attribution barely moves when one is removed (ratio +0.007); it points to

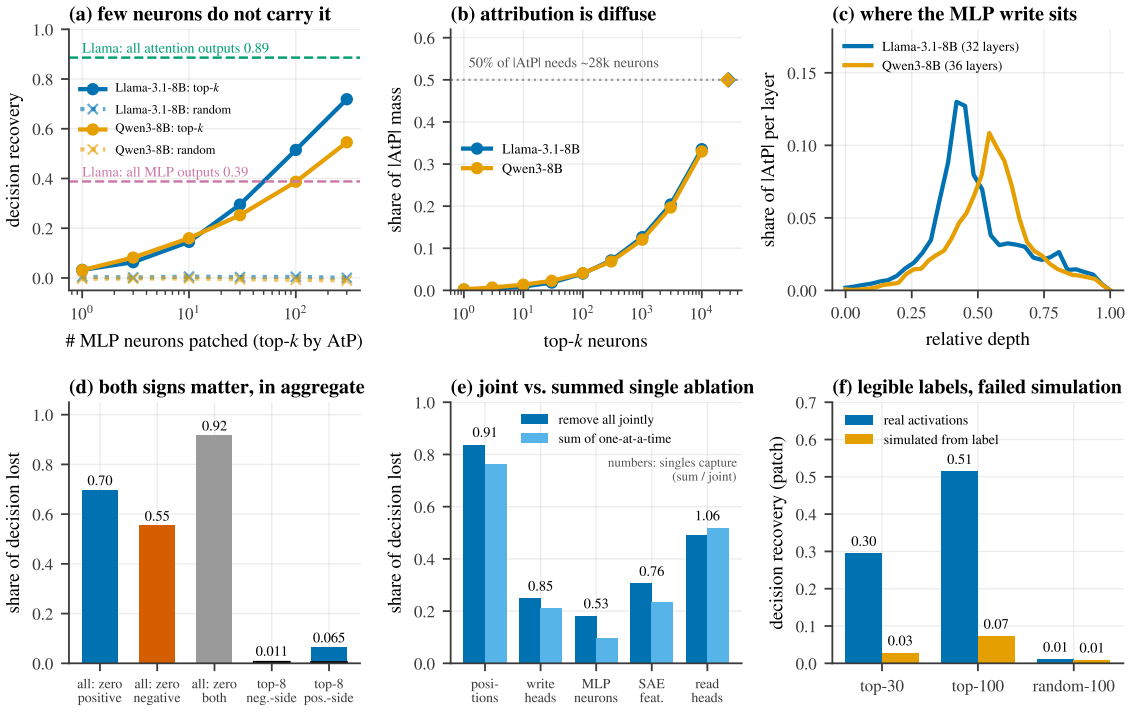


Figure 16: **The note at neuron level** (Llama-3.1-8B unless marked; 24 pairs). (a) Patching the top-k MLP neurons (by AtP) at the note positions from SAFE into UNSAFE recovers the decision gradually; random layer-matched neurons recover ≈ 0. Dashed: patching every attention output (green) or every MLP output (pink) at the same positions. (b) Cumulative share of |AtP| mass held by the top-k neurons; diamonds mark 50%. (c) Per-layer share of |AtP|. (d) Share of the decision lost when positive or negative activations are zeroed in both prompts, for all neurons and for the 8 most important negative-side and positive-side neurons (black ticks: sign-matched random neurons). (e) Removing a set of carriers jointly vs. summing one-at-a-time removals; numbers above the bars give the fraction of the joint effect that single removals capture. (f) Gao's simulation–substitution test: patching real activations vs. activations an LLM simulates from each neuron's label.

a threshold-like combination downstream, though backup outside the measured set is not ruled out. The margin is thin: the joint-minus-sum CI is $[-0.011, +0.178]$.

Legible labels, failed explanations. An LLM labeller (MiMo-v2.6-pro) labels each of the top-100 neurons from its 20 strongest positive and 15 strongest negative contexts in 2.5M tokens of FineWeb-Edu, blind to our task. It calls 26 "one job" and 70 "two related jobs", i.e. 96 of 100 read as clean. Legibility does not single them out: 100 layer-matched random neurons score the same (10 and 90). Apart from the top neuron (L25.11382, labelled "refusal / declining to comply"), the labels describe generic clause-boundary, continuation and connective detectors that say nothing about the conclusion. We then apply *Gao's simulation–substitution test* [7], the substitution form of simulation-based explanation scoring [2]: an explanation counts only if simulated activations can stand in for the real ones without degrading the model. Gao states the criterion qualitatively; the quantitative readout below (fraction of mean-ablation damage avoided) is ours. The same LLM predicts each neuron's per-token activation from its label alone (an integer in $[-10, 10]$, mapped to activation units by a per-neuron linear fit). We then replace the real activations with the simulated

Table 6: **Joint vs. one-at-a-time ablation** (Llama-3.1-8B, 24 pairs; pooled over pairs, 95% bootstrap CIs). Drops are shares of the decision lost on the clean run. Ratio $= 1$: additive; > 1: single ablations undercount; < 1: overlap. Sufficiency ratio: the same comparison for patching SAFE values into the UNSAFE run. The MLP necessity ratio is unstable (summed single drops near 0 in some pairs), so we report the singles' capture instead.

carrier	joint drop	Σ single drops	singles capture	suff. ratio (joint/Σ)
aggregator positions (16)	0.837	0.764	0.91 $[0.83, 0.99]$	0.79 $[0.73, 0.86]$
write heads (15)	0.249	0.212	0.85 $[0.59, 1.10]$	0.50 $[0.47, 0.54]$
MLP neurons (30)	0.181	0.097	0.53 $[-0.02, 1.06]$	0.80 $[0.66, 1.00]$
SAE L14 features (30)	0.305	0.233	0.76 $[0.51, 1.00]$	1.16 $[1.02, 1.32]$
read heads (12)	0.490	0.518	1.06 $[1.01, 1.11]$	0.84 $[0.81, 0.87]$

ones and measure how much of the mean-ablation damage this avoids. The labels fail (Figure 16f). The median correlation between simulated and real activations is 0.01 for the top-100 neurons (0.09 for random ones), and on general text the median neuron avoids none of the damage ($+0.00$). At the decision, simulated activations of the top 30 recover 0.03 of the decision against 0.30 for the real ones (top 100: 0.07 vs. 0.52), and substituting them does more damage than mean-ablating the neurons. Even the clean "refusal" neuron peaks on the token *before* the verb (it predicts "refused" rather than responding to it), which its label misses. This is the decode-vs.-cause gap of Section 3 once more: a unit can be named convincingly without the name predicting its behavior.

Do the read heads read neuron directions? Projecting each top neuron's output through a read head's OV circuit onto the safe-vs.-unsafe unembedding difference, 9 of the 12 read heads show no preference for note neurons over layer-matched random ones. Three of the latest heads (31.1, 30.26, 30.24) prefer them in some scenarios. The note is therefore re-encoded, through attention at the aggregators and in later layers, before the read heads pick it up; the read side does not tap individual neurons.

Scope. The full neuron-level analysis is on one model (Llama-3.1-8B); Qwen3-8B replicates only the attribution and patching steps. The neuron undercount is borderline at 95%. Labels and simulations come from a single LLM (MiMo-v2.6-pro) with an integer scale and a linear calibration, so a stronger explainer or simulator could narrow the gap, although real activations currently recover seven to ten times more of the decision than simulated ones.

F Editing: the baseline frontier and the K-sweep

Figure 17 plots the cost/correctness frontier behind Section 4: there is no single dominant method. `field+erratum` and the bare ERRATUM reach full correctness at $\sim$5–13% recompute with no prompt surgery; hoist-to-end matches them but rewrites the prompt; the in-place edit and a KV-deviation-ranked CacheBlend-style recompute are cheap but incorrect on these gated decisions. Numeric values are in Table 7. Figure 18 gives the full `field+selective@`K sweep across seven models, making the model-dependence of the minimal recompute explicit: small K suffices for the Qwen3 family but not for several others—the tool is effective but unreliable.

Figure 19 shows that the editing fix is an *attention-architecture* method: the erratum recovers the decision under reasoning on attention (GQA) and sliding-window backbones, partially on a hybrid attention+SSM model, and weakly on a pure SSM whose recurrent state has no per-token look-back.

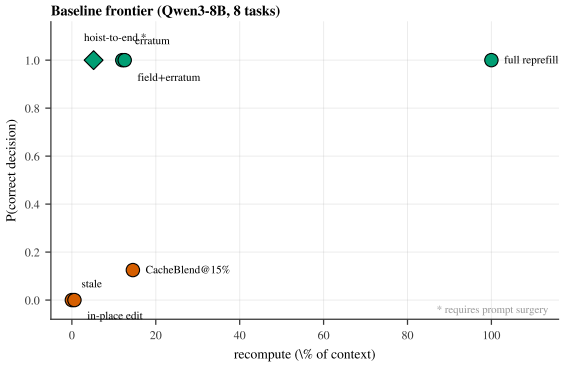


Figure 17: Cost/correctness frontier.

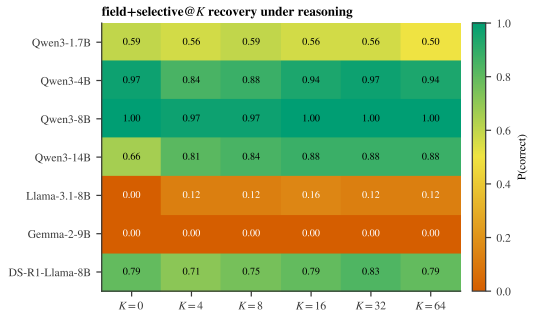


Figure 18: `field+selective@`K across models.

Table 7: Baseline frontier (Qwen3-8B, 8 gated tasks). "surgery" = requires rewriting the prompt.

method	P(correct)	recompute	prompt surgery
full reprefill	1.00	100%	no
hoist-to-end	1.00	5.2%	**yes**
`field+erratum`	1.00	12.6%	no
ERRATUM	1.00	12.0%	no
CacheBlend@15%	0.13	14.5%	no
`in_place` edit	0.00	0.6%	no
stale (reuse)	0.00	0%	no

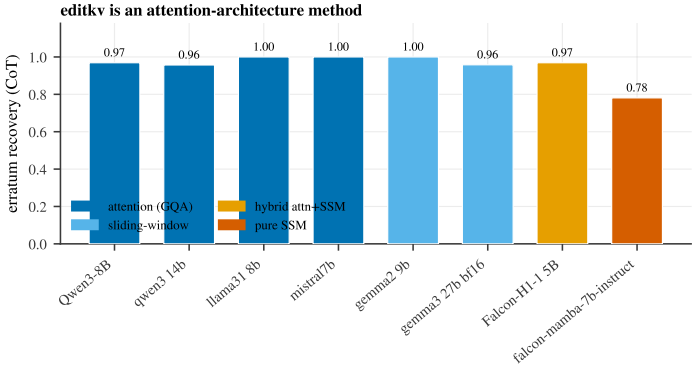


Figure 19: Erratum recovery under reasoning across attention, sliding-window, hybrid, and pure-SSM backbones.

G Composing: per-domain scorecards, multimodal, and the MLA adapter

Figure 20 is the composable scorecard: decision agreement between a transplanted skill and full recompute, by model and content type (facts in two insertion points, and agentic tool-calling). Standard attention models are at or near 1.0; the sliding-window Gemma models are the consistent exception (addressed in Section 8). Figure 22 breaks the multimodal result down by task category, and Figure 21 reports the MLA decoupled-`k_pe` adapter fidelity and the transplant TTFT speedup across models. Figure 23 diagrams the three attention-variant adapters of Section 8: what each

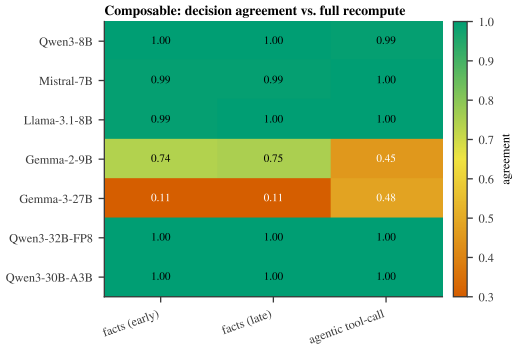


Figure 20: Composable agreement by model × content type.

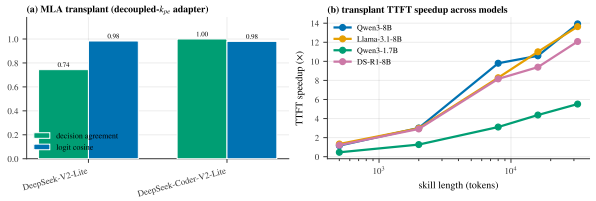


Figure 21: MLA adapter fidelity (a) and transplant TTFT speedup across models (b).

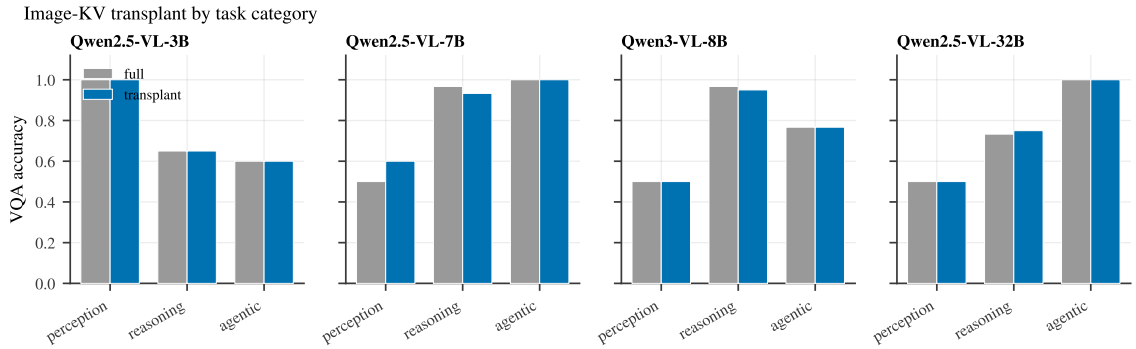


Figure 22: Image-KV transplant by task category (perception / reasoning / agentic), full re-encode vs. transplant, across four vision-language models. The transplant tracks full accuracy category-by-category.

representation caches, and exactly which slice of it the reposition touches.

H Long-horizon robustness and the cross-referential memory test

This appendix collects two stress-tests for the leave-stale+erratum scheme. The first asks whether per-edit errors *compound* over a long agent trajectory (they do not); the second asks whether splitting *cross-referential* facts across independently-precompiled blocks breaks a two-hop chain—the case Section 7's E4 left open (it does, and the mechanism explains why).

No compounding error over a long trajectory (Figure 24). A standing risk for any leave-stale scheme is that small per-edit errors *compound* over a long agent trajectory. We test it directly. A single gated field (a clearance level) toggles between a granting and a denying value every turn over a 28-turn trajectory; we maintain ONE evolving KV cache—reuse the static prefix forever, apply each state change as an appended erratum, never recompute the downstream—and compare its per-turn decision to a *full reprefill of the byte-identical token sequence* (errata included). The only difference between the two is whether downstream KV was recomputed after each change, so per-turn agreement isolates exactly the cost of leaving KV stale as a function of trajectory length. Across three families (Qwen3-8B, Llama-3.1-8B, Mistral-7B) the next-token *decision logits stay faithful*—cosine 0.987–0.999 with a flat first-third→last-third profile (e.g. 0.992 → 0.996 on Llama)—and the

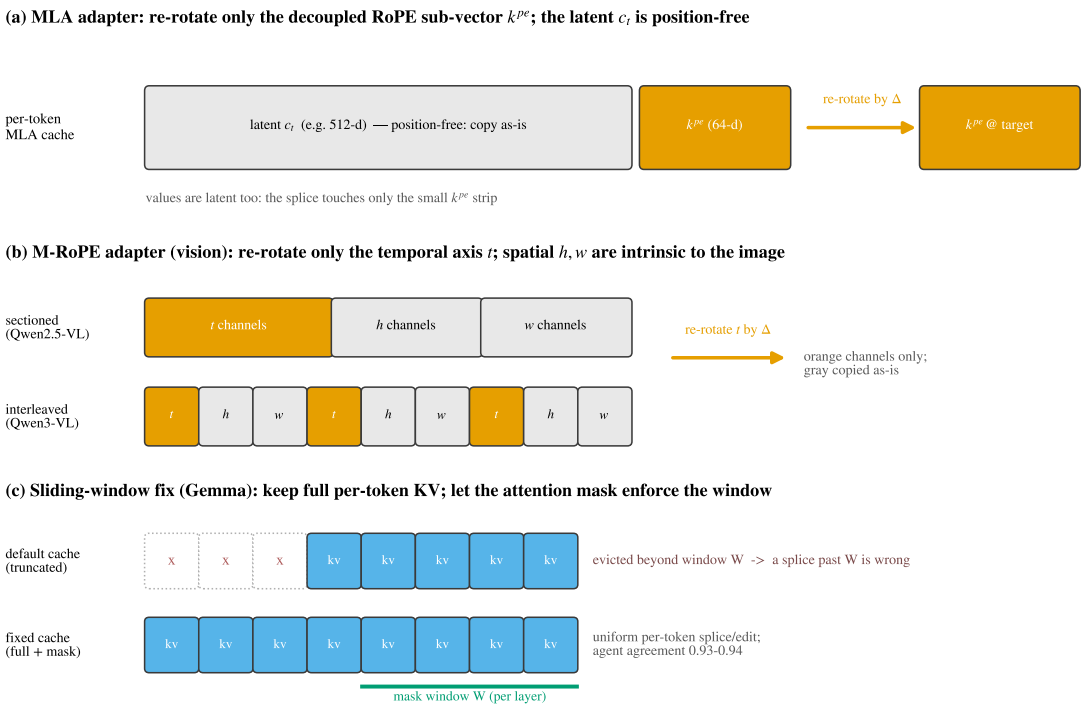


Figure 23: **The attention-variant adapters.** (a) MLA caches a position-free latent c_t plus a small decoupled-RoPE sub-vector `k_pe`; repositioning re-rotates only `k_pe` and copies the latent as-is. (b) M-RoPE factors the rotary channels into temporal/height/width axes—sectioned (Qwen2.5-VL) or interleaved (Qwen3-VL); moving an image within a trajectory re-rotates only the temporal channels, since the spatial axes are intrinsic to the image. (c) Sliding-window layers default to a window-truncated cache, which breaks any splice past W; keeping the full per-token KV and letting the attention mask enforce the window restores uniform edit/splice semantics.

agreement-vs-turn slope is within ± 0.01/turn (no systematic decline). Discrete decision agreement is high in aggregate (0.79–0.99) but noisier than the cosine, because these gated decisions sit near the action boundary (oracle accuracy 0.52–0.82), where a sub-percent logit difference can flip a discrete choice—the same boundary sensitivity noted for greedy CoT in Section 7. The leave-stale+erratum cache does not degrade with trajectory length: there is no compounding drift, only boundary noise.

Cross-referential facts (the Section 7 E4 test). E4 found splitting *independent* relevant facts across independently-precompiled blocks decision-lossless. We test the case it left open—a genuinely *cross-referential* chain. A memory contains a `DEFINITION` line A naming which setting governs the request, whose value B lives elsewhere; the decision needs the two-hop resolution $A \rightarrow B$. We lay A and B so they straddle a block boundary and compare a transplant that *splits* them (boundary between A and B) against one that keeps them *colocated* (boundary moved past both), holding everything else fixed. On Llama-3.1-8B—which performs this two-hop decision at full-recompute accuracy 0.85 (n=80, balanced)—splitting drops agreement with full recompute to 0.46 versus 0.76 colocated, a 0.30 penalty (25 vs. 1 discordant instances, McNemar $p = 8 \times 10^{-7}$); an *independent* fact pair split the same way costs only 0.04 (Mistral-7B). The mechanism explains it: block B precompiled in isolation never attended to its referent A, so the chain is not memoized and

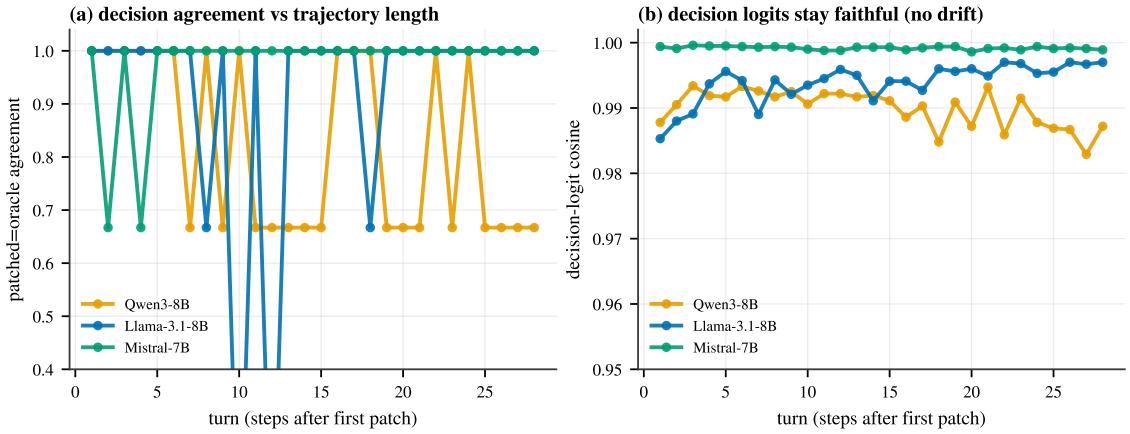


Figure 24: **No compounding error over a long trajectory.** A gated field toggles every turn for 28 turns; one evolving leave-stale+erratum cache vs. full reprefill of the identical text. (a) Per-turn decision agreement stays high with boundary noise (no downward trend). (b) The decision-logit cosine stays flat at 0.99+—no drift with trajectory length.

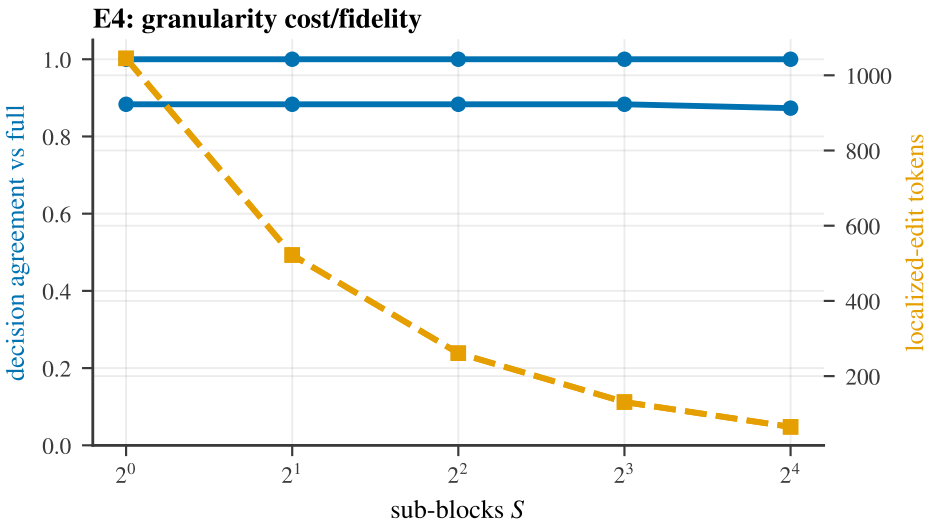


Figure 25: **E4 — granularity is a free knob** (Section 7). Splitting memory into S independently-precompiled blocks makes a localized edit $S\times$ cheaper and stays decision-lossless to $S{=}16$; only genuinely cross-referential facts must be kept in one block (cross-referential test above).

the splice cannot restore it. (Qwen3-8B is at chance on the direct two-hop decision—consistent with Section 7's observation that direct gated decisions are at chance for the reasoning-native family $\leq$32B—so it is uninformative here.) The guidance is simple: keep cross-referential facts within one precompiled block; independent facts may be split freely.

I The user-memory agent

The agent of Section 7 keeps the layout `[system][trajectory][MEMORY][query]`. The system prompt is prefilled once; the trajectory grows by cached deltas; the user-memory chunk is precompiled once in isolation and RoPE-repositioned to float just before the query each turn (an $O(L_{\text{mem}})$ re-rotation, no re-prefill) with one boundary token repaired; a memory change is applied either by recompiling the isolated chunk ($O(L_{\text{mem}})$, once) or by appending a salient erratum into the cached trajectory stream (append-only, composes with prefix caching). Confirmatory protocol: gated decisions whose governing facts live in a Markdown memory, balanced labels, chain-of-thought as the competent regime (direct one-shot memory-gated decisions are at chance for $\leq$8B models), with cluster-bootstrap CIs (10^4, persona-level), TOST equivalence (δ=0.03; cosine ≥ 0.98), GEE-logistic (cluster-robust), McNemar, and BH-FDR. Faithfulness is read against a token-matched full-reprefill oracle. Pre-registered hypotheses, margins, and the inclusion gate (oracle accuracy ≥ 0.80) are released with the code.

J Online serving and weight-editing methodology

Online vLLM serving (Figure 14a). vLLM V1 `AsyncLLMEngine`, CUDA graphs enabled (*not* `enforce_eager`), continuous batching, automatic prefix caching on, `bfloat16`, GPU memory utilization 0.85. Workload: a shared $\sim$8,066-token policy (real τ^2-bench retail policy plus neutral padding) with one mutable field; 96 requests per arm, 64 output tokens each. Requests arrive as a *Poisson* process at offered rates $\{2,4,8,16\}$ req/s and unthrottled (saturation). We timestamp first-token and completion per request in the client loop (TTFT, TPOT, end-to-end) and read each arm's APC hit-rate from the engine's Prometheus counters (`vllm:gpu_prefix_cache_hits`, `..._queries`). The in-prefix baseline writes the new field value early (invalidating downstream APC blocks); the erratum keeps the old prefix and appends the update. Headline: throughput speedup grows $1.6\times \rightarrow 14.5\times$ as load rises to saturation; p90 TTFT 53–398$\times$ lower; APC hit-rate 98.5% vs. 1.0%.

Weight editing (Table 1). ROME is implemented from scratch (uncentered key covariance $C=\mathbb{E}[kk^\top]$ at a mid MLP layer over a text sample, an optimized value $v^\star$, and the closed-form rank-one update of `down_proj`) and *validated on the canonical factual edit* ("the Eiffel Tower is in" *Paris*$\rightarrow$*Rome*, locality preserved) before use, so the baseline is faithful. LoRA fine-tunes (r=8, `q,v,down` projections) on the stale-context decision until it flips. All methods are evaluated on the same gated decision; cross-request contamination is measured over 8 held-out orders that are genuinely still pending (correct = cancel), and collateral over a 10-item battery of unrelated single-token gated decisions.